

MÉMOIRE DE PROJET DE FIN D'ÉTUDES (PFE) — 5ème ANNÉE

Filière : Ingénierie Informatique & Intelligence Artificielle

**DÉVELOPPEMENT D'UN SYSTÈME DE DÉTECTION
D'INTRUSIONS RÉSEAU (NIDS) BASÉ SUR LE DEEP
LEARNING**

ÉTUDIANT

AKAKPO-DJAKPATA Atsou Mathieu

Élève Ingénieur en Cybersécurité & IA

MAÎTRE DE STAGE

M. MOUNCIF Hamza

3D Smart Factory

ENCADRANTE PFE

Mme. HAIDRAR Saida

Encadrante PFE — HESTIM

ENTREPRISE D'ACCUEIL

3D Smart Factory

Casablanca, Maroc

Dataset

CICIDS2017 — 2,83 M lignes, 77 features, 15
classes

Modèles Évalués

11 architectures Deep
Learning

Période de Stage

Février 2026 — Juin
2026

Année Universitaire 2025-2026

DÉDICACES

À mes parents,

Qui m'ont soutenu à chaque étape de mon parcours académique et personnel, en m'inculquant le goût de l'effort, de la rigueur et de la persévérance. Ce travail est le fruit de vos sacrifices quotidiens et de votre confiance inconditionnelle. Chaque heure passée à étudier, à coder et à rédiger a été portée par votre amour et votre soutien moral indéfectible.

À mes enseignants de HESTIM,

Qui ont su guider mes premiers pas dans le monde fascinant des technologies de l'information, de la cybersécurité et de l'intelligence artificielle. Vos cours, votre patience pédagogique et votre passion pour la transmission du savoir m'ont fourni les bases théoriques et pratiques nécessaires à la réalisation de ce projet ambitieux.

À mes collègues et amis de promotion,

Pour les échanges stimulants, les nuits de code partagées, les encouragements mutuels dans les moments de doute et les instants de partage intellectuel qui ont jalonné cette belle année universitaire. Votre présence a rendu ce parcours exigeant non seulement supportable, mais véritablement épanouissant.

À tous ceux qui croient que la technologie, mise au service de la sécurité, peut contribuer à construire un espace numérique plus sûr et plus équitable pour tous,

Je dédie ce mémoire de projet de fin d'études.

REMERCIEMENTS

Je tiens tout d'abord à exprimer ma profonde gratitude à mon encadrante de projet de fin d'études, Madame HAIDRAR Saida, pour son encadrement exceptionnel, sa disponibilité constante, ses conseils précieux et sa rigueur scientifique qui ont grandement contribué à la structuration et au succès de cette recherche appliquée. Ses retours constructifs, sa capacité à poser les bonnes questions au bon moment et sa vision globale de la recherche en cybersécurité m'ont appris à allier rigueur académique et pragmatisme professionnel. Chaque réunion de suivi a été l'occasion d'affiner mes hypothèses, de challenger mes résultats et d'élever le niveau d'exigence de ce travail.

Je remercie également chaleureusement mon maître de stage au sein de 3D Smart Factory (CSIT), Monsieur MOUNCIF Hamza, pour sa confiance, son accueil chaleureux au sein de l'équipe et pour m'avoir donné l'opportunité de travailler sur des problématiques de cybersécurité réelles et complexes. Son expertise sur les infrastructures de production cloud et ses retours d'expérience terrain sur les contraintes concrètes liées au déploiement de systèmes d'intelligence artificielle dans des environnements d'entreprise hautement sécurisés ont été précieux pour orienter les choix architecturaux de ce projet.

Je remercie l'entreprise 3D Smart Factory (CSIT), dont la mission d'innovation technologique au service des entreprises industrielles marocaines et africaines constitue un cadre stimulant. L'accès à l'infrastructure, aux outils et à la culture DevSecOps de l'entreprise a joué un rôle déterminant dans la réussite de la phase de déploiement cloud de ce projet.

Mes sincères remerciements s'adressent à l'ensemble du corps professoral et administratif de l'école HESTIM pour la qualité de l'enseignement prodigué tout au long de mon cursus d'ingénieur de cinq années. Les compétences acquises en cybersécurité, en systèmes d'information, en algorithmique, en mathématiques appliquées et en intelligence artificielle ont été les piliers de la réalisation de ce projet.

Enfin, je remercie les membres du jury pour l'honneur qu'ils me font en acceptant d'évaluer ce travail et d'apporter leur regard expert sur les résultats présentés dans ce mémoire. Leurs questions et remarques me permettront de consolider davantage cette recherche.

LISTE DES ACRONYMES

Sigle	Signification	Sigle	Signification
AI / IA	Artificial Intelligence / Intelligence Artificielle	IPS	Intrusion Prevention System — Système de prévention d'intrusions
API	Application Programming Interface — Interface de programmation applicative	IQR	Interquartile Range — Écart interquartile
APT	Advanced Persistent Threat — Menace persistante avancée	LSTM	Long Short-Term Memory — Réseau de neurones récurrent à mémoire à long terme
AUC-ROC	Area Under the Curve – Receiver Operating Characteristic	MLP	Multi-Layer Perceptron — Perceptron multicouche
AWS	Amazon Web Services — Fournisseur d'infrastructure cloud leader mondial	NIDS	Network Intrusion Detection System — Système de détection d'intrusions réseau
BiLSTM	Bidirectional Long Short-Term Memory — Réseau récurrent bidirectionnel	OWASP	Open Web Application Security Project
C&C / C2	Command & Control — Serveur de contrôle à distance de botnets	RDP	Remote Desktop Protocol — Protocole de bureau à distance
CIC	Canadian Institute for Cybersecurity — Institut canadien de la cybersécurité	SMOTE	Synthetic Minority Over-sampling Technique
CNN	Convolutional Neural Network — Réseau de neurones convolutif	SOAR	Security Orchestration, Automation, and Response
DDoS	Distributed Denial of Service — Déni de service distribué	SOC	Security Operations Center — Centre opérationnel de sécurité
DL	Deep Learning — Apprentissage profond	SQS	Simple Queue Service — File de messages managée AWS
DoS	Denial of Service — Déni de service	SSH	Secure Shell — Protocole d'accès distant sécurisé
ENI	Elastic Network Interface — Interface réseau virtuelle AWS	TCP	Transmission Control Protocol — Protocole de contrôle de transmission
FTP	File Transfer Protocol — Protocole de transfert de fichiers	TCN	Temporal Convolutional Network — Réseau convolutif temporel
GRU	Gated Recurrent Unit — Unité récurrente à portes	UDP	User Datagram Protocol — Protocole de datagramme utilisateur

HTTP	HyperText Transfer Protocol — Protocole de transfert hypertexte	VPC	Virtual Private Cloud — Réseau virtuel privé sur le cloud
IaC	Infrastructure as Code — Infrastructure sous forme de code	VXLAN	Virtual Extensible LAN — Protocole d'encapsulation réseau
ICMP	Internet Control Message Protocol — Protocole de messages de contrôle Internet	XAI	Explainable Artificial Intelligence — Intelligence artificielle explicable
IDS	Intrusion Detection System — Système de détection d'intrusions	XSS	Cross-Site Scripting — Injection de scripts malveillants côté client

Table des matières

DÉDICACES	2
REMERCIEMENTS	3
LISTE DES ACRONYMES	4
RÉSUMÉ	11
ABSTRACT.....	13
INTRODUCTION GÉNÉRALE	15
1. Contexte et enjeux.....	15
2. Problématique	15
3. Objectifs du projet.....	16
4. Structure du mémoire.....	16
CHAPITRE 1 : Présentation de l'Entreprise, Cadrage du Projet et État de l'Art.....	18
1.1. Présentation de l'Entreprise d'Accueil : 3D Smart Factory	18
1.2. Contexte et Périmètre du Projet	19
1.3. Cartographie Détaillée des Menaces Réseau Contemporaines	19
1.3.1. Évolution du Paysage des Menaces 2020-2026.....	19
1.3.2. Anatomie d'une Attaque Réseau Moderne.....	19
1.3.3. Statistiques des Cyberattaques Mondiales	20
1.4. Analyse des Besoins	21
1.4.1. Besoins Fonctionnels	21
1.4.2. Besoins Non Fonctionnels	21
1.5. Acteurs et Parties Prenantes du Projet	22
1.6. Stack Technologique.....	22
1.6.1. Machine Learning & Deep Learning	22
1.6.2. Infrastructure Cloud & Déploiement	23
1.6.3. Backend & Frontend	23
1.7. Méthodologie de Gestion de Projet.....	23
1.8. Paysage des Menaces Cybernétiques Actuelles.....	24
1.8.1. Taxonomie des Cyberattaques Modernes	24
1.8.2. Le Défi du Trafic Chiffré	25
1.9. Limites des Approches de Détection Traditionnelles	25
1.9.1. Détection par Signatures	25
1.9.2. Détection par Anomalies Statistiques Classiques	26
1.10. Fondements Théoriques du Deep Learning pour la Détection d'Intrusions.....	26
1.10.1. Le Perceptron Multicouche (MLP)	26
1.10.2. Le Mécanisme d'Attention sur les Caractéristiques	26
1.10.3. Les Réseaux Récurrents (LSTM, BiLSTM, GRU).....	27

1.11. Le Dataset CICIDS2017 : Description et Caractéristiques	27
1.12. Revue de la Littérature : Deep Learning appliqué aux NIDS	28
1.12.1. Yin et al. (2021) — LSTM Récurrent pour la Classification Binaire.....	29
1.12.2. Andresini et al. (2022) — Autoencoder Hybride sur CICIDS2017.....	29
1.12.3. Zhang et al. (2022) — CNN 1D avec Focal Loss.....	29
1.12.4. Ferrag et al. (2023) — Transformer pour la Détection d'Intrusions IoT	29
1.12.5. Yang et al. (2023) — BiLSTM avec Attention.....	29
1.13. Positionnement Unique et Hypothèses de Travail	30
1.14. Architecture Détaillée des Systèmes NIDS : Topologies et Positionnement	31
1.14.1. NIDS vs HIDS vs Hybrides	31
1.14.2. Modes de Déploiement d'un NIDS	31
1.15. Analyse Mathématique des Métriques d'Évaluation.....	32
1.15.1. Matrice de Confusion Multi-Classes	32
1.15.2. Justification du Choix du F1-Score Macro	32
1.16. Analyse de la Sécurité Cloud : Modèle de Responsabilité Partagée AWS.....	32
1.16.1. Modèle de Responsabilité Partagée	33
1.16.2. Groupes de Sécurité et Isolation Réseau.....	33
1.17. Étude Comparative des Frameworks et Datasets pour les NIDS.....	34
1.17.1. Comparatif des Principaux Datasets NIDS.....	34
1.17.2. Justification du Choix de PyTorch vs TensorFlow/Keras	34
Conclusion du Chapitre 1.....	35
CHAPITRE 2 : Prétraitement de Données, Modélisation et Benchmark Quantitatif.....	36
2.1. Pipeline de Prétraitement des Données.....	36
2.1.1. Chargement et Consolidation du Dataset.....	36
2.1.2. Traitement des Valeurs Manquantes et Aberrantes	36
2.1.3. Encodage et Standardisation	36
2.1.4. Stratégie de Séparation Train / Test Étanche	37
2.1.5. Rééchantillonnage Hybride SMOTE	37
2.1.6. Code du Pipeline de Prétraitement.....	37
2.1.7. Analyse Statistique des Caractéristiques Discriminantes	38
2.2. Architecture des 11 Modèles de Deep Learning Évalués	39
2.2.1. Modèle 1 : MLP Standard (Baseline)	39
2.2.2. Modèle 2 : CNN 1D.....	39
2.2.3. Modèle 3 : ResNet 1D.....	39
2.2.4. Modèle 4 : TCN (Temporal Convolutional Network)	40
2.2.5. Modèle 5 : LSTM.....	40
2.2.6. Modèle 6 : BiLSTM.....	40

2.2.7. Modèle 7 : GRU	40
2.2.8. Modèle 8 : CNN-LSTM Hybride.....	40
2.2.9. Modèle 9 : Autoencoder Hybride	41
2.2.10. Modèle 10 : Transformer	41
2.2.11. Modèle 11 : Attention MLP (Notre Contribution).....	41
2.2.11. Implémentation PyTorch du Modèle Attention MLP	42
2.2.12. Analyse du Mécanisme d'Attention par Classe d'Attaque	42
2.3. Protocole Expérimental d'Entraînement	43
2.4. Métriques d'Évaluation Multi-Classes	44
2.5. Résultats du Benchmark Quantitatif	44
2.6. Analyse Critique des Résultats	44
2.6.1. Supériorité de l'Attention MLP	45
2.6.2. Inadaptation du Transformer aux Données Tabulaires	45
2.6.3. Analyse des Confusions Résiduelles	45
2.6.4. Implications Opérationnelles pour le SOC	46
2.6.5. Latences d'Inférence et Viabilité Production	46
2.6.7. Dynamique d'Entraînement de l'Attention MLP.....	46
2.6.8. Comparaison des Vitesses de Convergence.....	47
Conclusion du Chapitre 2.....	48
CHAPITRE 3 : Architecture Système en Production (AWS CyberRange) et Validation.....	49
3.1. Architecture Globale du CyberRange AWS	49
3.1.1. Philosophie de Conception	49
3.1.2. Ressources AWS Provisionnées	49
3.1.3. Déclaration de l'Infrastructure Réseau (vpc.tf)	51
3.1.4. Playbook de Configuration de la Sonde IDS	52
3.1.5. Méthodologie des Tests de Charge	53
3.2. Mécanisme de Capture Passive : VPC Traffic Mirroring et VXLAN	54
3.2.1. Fonctionnement du VPC Traffic Mirroring	54
3.2.2. Avantages Opérationnels de l'Architecture Hors Bande.....	54
3.3. Démon d'Inférence Temps Réel : ids_engine.py	55
3.3.1. Architecture du Démon.....	55
3.3.2. Configuration du Démon d'Inférence (ids_engine.py)	55
3.3.3. Justification du Seuil de Confiance à 0,40	56
3.4. Backend FastAPI et Résolveur d'Adresses AWS	56
3.4.1. Architecture du Backend.....	56
3.4.2. Diffusion WebSocket vers le Dashboard React.....	57
3.5. Dashboard SOC React	57

3.5.1. Architecture Front-End	57
3.5.2. Architecture des Composants React	58
3.5.3. Expérience Utilisateur et Accessibilité	58
3.6. Validation Systémique par Simulation d'Attaques	60
3.6.1. Scénarios de Simulation.....	60
3.6.2. Synthèse des Latences Bout-en-Bout.....	60
3.7. Analyse de Sécurité de l'Architecture Déployée.....	61
3.7.1. Vecteurs d'Attaque Potentiels contre le NIDS	61
3.7.2. Mécanismes de Résilience Intégrés	61
3.7.3. Positionnement par Rapport aux Solutions du Marché.....	62
Conclusion du Chapitre 3.....	63
CONCLUSION GÉNÉRALE ET PERSPECTIVES	64
Bilan des réalisations	64
Perspectives Industrielles.....	64
1. Réponse Active Automatisée (SOAR).....	64
2. Explicabilité des Décisions (XAI)	65
3. Détection d'Anomalies Zero-Day Hybride	65
4. Adaptation à l'IoT et aux Réseaux OT	65
5. Entraînement Fédéré pour la Confidentialité	65
RÉFÉRENCES BIBLIOGRAPHIQUES.....	67
ANNEXES — GLOSSAIRE TECHNIQUE	68

LISTE DE TABLEAUX

Tableau 1: Statistiques clés du paysage des cybermenaces mondiales (Sources : IBM Security, Verizon DBIR, Cloudflare, 2024).....21

Tableau 2: Matrice des acteurs et parties prenantes du projet22

Tableau 3: Distribution complète des 15 classes du dataset CICIDS201728

Tableau 4: Synthèse comparative de l'état de l'art et positionnement du projet30

Tableau 5: Comparaison des modes de déploiement d'une sonde NIDS31

Tableau 6: Comparatif des principaux datasets pour la détection d'intrusions réseau.....34

Tableau 7: Top 10 des caractéristiques les plus discriminantes du dataset CICIDS2017 (Random Forest Feature Importance)39

Tableau 8: Protocole expérimental standardisé pour le benchmark des 11 modèles.....43

Tableau 9: Résultats complets du benchmark des 11 architectures de Deep Learning sur CICIDS201744

Tableau 10 : Comparaison des latences d'inférence et viabilité de déploiement en production.46

Tableau 11: Comparaison des dynamiques de convergence des architectures principales47

Tableau 12: Résultats des tests de charge de la sonde IDS sur instance t3.large54

Tableau 13: Résultats des simulations d'attaques et performances de détection en production .60

Tableau 14: Comparaison de notre NIDS-AI avec les principales solutions du marché.....62

LISTE DE FIGURES

Figure 1: Planning prévisionnel24

Figure 2: Architecture Réseau du CyberRange51

Figure 3: Dashboard de l'application de surveillance des alertes59

Figure 4: Diagrammes analytiques de l'application de surveillance des alertes59

RÉSUMÉ

La prolifération exponentielle des cyberattaques sophistiquées, ciblant indistinctement des infrastructures critiques, des entreprises privées et des institutions gouvernementales, impose le développement de systèmes de détection d'intrusions réseau (NIDS) de nouvelle génération. Les approches classiques fondées sur la correspondance de signatures statiques — telles que Suricata ou Snort — révèlent des limites structurelles profondes : incapacité à détecter les menaces inédites (zero-day), aveuglement face au trafic chiffré (TLS 1.3) et génération de volumes d'alertes non exploitables conduisant à la fatigue des analystes de sécurité (SOC).

Ce mémoire de projet de fin d'études présente la conception, l'évaluation comparative et le déploiement en conditions réelles d'un NIDS de nouvelle génération intégralement fondé sur l'apprentissage profond (Deep Learning). Un benchmark méthodiquement standardisé a été conduit sur 11 architectures de réseaux de neurones profonds — incluant MLP, CNN 1D, ResNet 1D, TCN, LSTM, BiLSTM, GRU, CNN-LSTM, Autoencoder hybride, Transformer et notre modèle original Attention MLP — en utilisant le dataset académique de référence CICIDS2017 (2 830 743 flux réseau bidirectionnels, 77 caractéristiques statistiques, 15 classes d'attaques distinctes).

Pour surmonter le déséquilibre extrême de classes inhérent au dataset (le trafic normal représente plus de 82 % des échantillons), un pipeline de prétraitement hybride rigoureux a été mis en oeuvre, combinant l'écrêtage interquartile (IQR) pour la suppression des valeurs aberrantes, la normalisation Z-Score pour l'homogénéisation des échelles et la technique SMOTE (Synthetic Minority Over-sampling Technique) pour la création d'exemples synthétiques des classes d'attaques minoritaires.

Les résultats expérimentaux démontrent que l'architecture originale Attention MLP surclasse l'ensemble des 10 architectures concurrentes, avec une exactitude (accuracy) de 98,28 %, un F1-Score macro-moyenné de 98,29 %, un AUC-ROC de 99,95 %, tout en étant le modèle le plus compact avec seulement 49 915 paramètres entraînaibles et un temps d'inférence remarquable de 0,42 milliseconde par flux réseau sur CPU standard.

Ce modèle optimal a été intégré dans un CyberRange cloud résistant à la charge, entièrement provisionné sur AWS (région eu-west-1) par Infrastructure as Code (Terraform) et configuré

automatiquement par Ansible. La capture passive et non intrusive du trafic de production est réalisée par le service natif AWS VPC Traffic Mirroring avec encapsulation VXLAN. Les alertes d'intrusion sont collectées asynchronement via Amazon SQS, persistées dans DynamoDB, puis diffusées en temps réel vers un dashboard SOC interactif développé en React. La validation systémique par simulation active de cyberattaques (SYN Flood, PortScan, ICMP Flood) démontre une efficacité de détection de 100 % avec un temps de réponse global de bout-en-bout inférieur à 2 secondes.

Mots-clés : *NIDS, Cybersécurité, Deep Learning, Attention MLP, CICIDS2017, AWS, VXLAN, VPC Traffic Mirroring, SOC Dashboard, Terraform, Ansible, PyTorch, SMOTE.*

ABSTRACT

The exponential proliferation of sophisticated cyberattacks targeting critical infrastructure, private companies, and government institutions worldwide demands the development of next-generation Network Intrusion Detection Systems (NIDS). Traditional signature-based detection engines — such as Suricata or Snort — exhibit deep structural limitations: an inability to detect novel unknown threats (zero-day attacks), complete blindness to encrypted traffic (TLS 1.3), and the generation of unmanageable alert volumes that lead to analyst fatigue in Security Operations Centers (SOC).

This final-year engineering thesis presents the design, comparative evaluation, and real-world deployment of a next-generation deep learning-based NIDS. A rigorously standardized benchmark was conducted across 11 deep neural network architectures — including MLP, 1D CNN, 1D ResNet, TCN, LSTM, BiLSTM, GRU, CNN-LSTM, Hybrid Autoencoder, Transformer, and our original Attention MLP — using the reference academic dataset CICIDS2017 (2,830,743 bidirectional network flows, 77 statistical features, 15 distinct attack classes).

To address the extreme class imbalance inherent to the dataset (normal traffic exceeds 82% of all samples), a rigorous hybrid preprocessing pipeline was deployed, combining Interquartile Range (IQR) clipping for outlier removal, Z-Score standardization for feature scaling, and the Synthetic Minority Over-sampling Technique (SMOTE) for creating synthetic examples of minority attack classes.

Experimental results demonstrate that our original Attention MLP architecture outperforms all 10 competing architectures, achieving 98.28% accuracy, a macro-averaged F1-Score of 98.29%, an AUC-ROC of 99.95%, while remaining the most compact model with only 49,915 trainable parameters and a remarkable inference latency of 0.42 milliseconds per network flow on standard CPU hardware.

The optimal model was integrated into a load-resistant cloud CyberRange, fully provisioned on AWS (eu-west-1 region) via Infrastructure as Code (Terraform) and automatically configured through Ansible. Passive, non-intrusive capture of production traffic is achieved using the native AWS VPC Traffic Mirroring service with VXLAN encapsulation. Intrusion alerts are collected

asynchronously via Amazon SQS, persisted in DynamoDB, then streamed in real-time to a custom interactive React SOC dashboard. Systemic validation through active cyberattack simulation (SYN Flood, PortScan, ICMP Flood) demonstrates 100% detection efficacy with an end-to-end latency of less than 2 seconds.

Keywords: *NIDS, Cybersecurity, Deep Learning, Attention MLP, CICIDS2017, AWS, VXLAN, VPC Traffic Mirroring, SOC Dashboard, Terraform, Ansible, PyTorch, SMOTE.*

INTRODUCTION GÉNÉRALE

1. Contexte et enjeux

La transformation numérique accélérée des organisations publiques et privées a conduit à une exposition croissante de leurs systèmes d'information aux menaces cyber. En 2024, le coût mondial de la cybercriminalité a dépassé les 9 000 milliards de dollars selon les estimations du cabinet Cybersecurity Ventures, plaçant les attaques informatiques comme l'une des principales menaces économiques mondiales, devant les catastrophes naturelles. Les données sensibles des entreprises, les infrastructures critiques (réseaux électriques, systèmes de santé, plateformes bancaires) et les informations personnelles des citoyens sont quotidiennement ciblées par des acteurs malveillants de plus en plus sophistiqués.

Face à cette menace protéiforme, la détection et la réponse aux incidents constituent la ligne de défense la plus critique des équipes de sécurité opérationnelles (SOC). Cependant, les systèmes de détection d'intrusions réseau (NIDS) traditionnels, qui reposent sur la correspondance de signatures statiques et de règles expertes définies manuellement, peinent à suivre le rythme d'évolution des attaques modernes. Les techniques d'obfuscation, le chiffrement généralisé du trafic réseau avec TLS 1.3 et l'émergence de variantes polymorphes d'attaques connues rendent ces approches structurellement inefficaces contre les menaces zero-day.

C'est dans ce contexte de rupture technologique que l'apprentissage profond (Deep Learning) s'impose comme un paradigme de rupture pour la cybersécurité défensive. En apprenant des représentations comportementales complexes directement à partir des données de flux réseau, les modèles d'apprentissage profond peuvent détecter des patterns malveillants inédits, résister au chiffrement du trafic applicatif et s'adapter dynamiquement à l'évolution du paysage des menaces.

2. Problématique

Malgré les avancées significatives publiées dans la littérature académique sur l'application du Deep Learning aux NIDS, un fossé opérationnel profond sépare les expériences de laboratoire des déploiements industriels. La grande majorité des travaux évaluent des architectures de réseaux de neurones complexes (Transformers, BiLSTMs) sur des jeux de données statiques,

sans se soucier de la latence d'inférence réelle, de la consommation mémoire ou de l'intégration dans une infrastructure de production.

La problématique centrale de ce projet est la suivante : Comment concevoir, entraîner et déployer en production un NIDS basé sur le Deep Learning qui soit simultanément précis (F1-Score > 98%), léger (< 50 000 paramètres), ultra-rapide (latence d'inférence < 1 ms) et opérationnellement intégré dans une architecture cloud de surveillance temps réel ?

3. Objectifs du projet

Ce projet de fin d'études poursuit quatre objectifs scientifiques et techniques complémentaires :

1. Réaliser un benchmark exhaustif et méthodiquement rigoureux de 11 architectures de Deep Learning sur le dataset CICIDS2017, en respectant les principes d'étanchéité des données pour éviter tout biais d'évaluation.
2. Concevoir et valider une architecture originale Attention MLP, capable de surpasser les modèles lourds de l'état de l'art en matière de classification multi-classes sur des flux réseau statistiques tout en restant déployable en production sans GPU dédié.
3. Développer un pipeline de prétraitement hybride SMOTE robuste qui corrige le déséquilibre extrême de classes du dataset sans induire de surapprentissage ni de data leakage.
4. Concevoir, déployer et valider une architecture système complète de CyberRange cloud sur AWS, intégrant la capture passive VPC Traffic Mirroring, l'inférence temps réel et un dashboard de supervision SOC interactif.

4. Structure du mémoire

Ce mémoire est organisé en trois chapitres principaux, encadrés d'une introduction et d'une conclusion générales :

- Le Chapitre 1 présente l'entreprise d'accueil, le cadrage méthodologique du projet, l'analyse de l'existant et l'état de l'art approfondi sur les approches de détection d'intrusions par apprentissage profond.
- Le Chapitre 2 détaille le pipeline de prétraitement des données, la modélisation mathématique des 11 architectures évaluées et le benchmark quantitatif complet avec analyse critique des résultats.

- Le Chapitre 3 décrit l'architecture système AWS CyberRange déployée en production, les mécanismes de capture passive réseau, le démon d'inférence temps réel et la validation systémique par simulation d'attaques.

CHAPITRE 1 : Présentation de l'Entreprise, Cadrage du Projet et État de l'Art

Ce premier chapitre constitue le socle contextuel et analytique de l'ensemble du mémoire. Il débute par une présentation détaillée de l'entreprise d'accueil, 3D Smart Factory (CSIT), et du cadre opérationnel dans lequel s'inscrit ce projet. Il pose ensuite les fondements théoriques et technologiques de la sécurité réseau, examine en profondeur les limites des approches de détection traditionnelles, et développe une revue critique de la littérature scientifique récente sur l'application du Deep Learning à la détection d'intrusions.

1.1. Présentation de l'Entreprise d'Accueil : 3D Smart Factory

3D Smart Factory, filiale opérationnelle du groupe CSIT (Centre des Solutions d'Intelligence Technologique), est une entreprise marocaine spécialisée dans l'innovation technologique à destination des secteurs industriels et tertiaires. Localisée à Casablanca, au carrefour des échanges économiques maghrébins, l'entreprise se positionne comme un acteur de référence dans les domaines de l'ingénierie logicielle, de la transformation digitale, de la cybersécurité et de l'intelligence artificielle appliquée.

La mission principale de 3D Smart Factory est d'accompagner les entreprises industrielles et tertiaires marocaines et africaines dans leur transformation numérique, en leur fournissant des solutions sur-mesure alliant performance technologique, sécurité by-design et intelligence des données. L'entreprise dispose d'une équipe pluridisciplinaire réunissant des ingénieurs en développement logiciel, des experts en infrastructure cloud, des spécialistes en cybersécurité et des data scientists, créant un environnement de travail propice à l'innovation et au dépassement technique.

Dans le domaine de la cybersécurité, 3D Smart Factory accompagne ses clients dans la mise en conformité réglementaire (ISO 27001, RGPD, loi marocaine 09-08), la mise en place d'architectures de sécurité (Zero Trust, Defense in Depth), la réalisation d'audits de vulnérabilités et la création de SOC managés. C'est dans ce contexte opérationnel concret que s'inscrit la problématique de ce PFE : concevoir un système de détection d'intrusions réseau de nouvelle génération, déployable dans les environnements cloud hybrides que gère l'entreprise pour le compte de ses clients.

1.2. Contexte et Périmètre du Projet

Le périmètre du projet couvre l'ensemble du cycle de vie d'un système de détection d'intrusions réseau basé sur l'apprentissage profond : depuis l'analyse des besoins et la revue de la littérature, en passant par l'acquisition et le prétraitement des données, la conception et le benchmark des architectures neuronales, jusqu'au déploiement en production sur une infrastructure cloud réaliste et sa validation systémique.

Le projet s'articule autour de deux axes complémentaires : un axe Recherche & Développement centré sur l'innovation algorithmique (conception du modèle Attention MLP, pipeline SMOTE hybride, benchmark comparatif) et un axe Ingénierie Système centré sur le déploiement opérationnel (CyberRange AWS, capture VPC Traffic Mirroring, dashboard SOC temps réel).

1.3. Cartographie Détaillée des Menaces Réseau Contemporaines

1.3.1. Évolution du Paysage des Menaces 2020-2026

Le paysage des cybermenaces a connu une transformation radicale au cours des six dernières années, accélérée par la pandémie de COVID-19 et la généralisation du télétravail. Selon le rapport Verizon Data Breach Investigations Report (DBIR) 2024, 83% des violations de données impliquent des acteurs externes, et 74% utilisent le facteur humain comme vecteur initial. Les attaques par ransomware ont augmenté de 413% entre 2019 et 2024, devenant la principale menace pour les organisations de toutes tailles.

La sophistication croissante des outils d'attaque, combinée à la démocratisation du modèle Cybercrime-as-a-Service (CaaS), a considérablement abaissé la barrière technique pour les acteurs malveillants. Des plateformes DDoS-for-hire permettent à des individus sans expertise technique de lancer des attaques volumétriques massives pour quelques dizaines de dollars. Cette démocratisation rend la détection comportementale d'autant plus critique : les signatures statiques ne peuvent capturer l'infinité des variantes générées par ces outils automatisés.

1.3.2. Anatomie d'une Attaque Réseau Moderne

Une cyberattaque réseau moderne suit généralement la séquence en sept phases du modèle Cyber Kill Chain de Lockheed Martin :

1. **Reconnaissance** : L'attaquant collecte des informations sur la cible via des sources ouvertes (OSINT), des scans passifs DNS et des analyses Shodan. Cette phase génère peu ou pas de trafic réseau anormal, rendant la détection très difficile.
2. **Armement** : Préparation des outils d'exploitation (exploit + payload + déployeur). Phase entièrement hors ligne, invisible pour les NIDS.
3. **Livraison** : Transmission du vecteur d'attaque vers la cible via email (phishing), web (drive-by download), réseau (exploitation directe). C'est la première phase visible par les NIDS réseau.
4. **Exploitation** : Déclenchement de la vulnérabilité (buffer overflow, SQL injection, CVE). Génère des patterns de trafic anormaux détectables.
5. **Installation** : Déploiement d'un accès persistant (backdoor, web shell, service système). Les communications C&C sont typiquement chiffrées mais présentent des patterns comportementaux caractéristiques.
6. **Commande et Contrôle (C2)** : Établissement d'un canal de communication entre l'attaquant et l'hôte compromis. Les balises C&C périodiques (beaconing) sont détectables par analyse statistique des intervalles d'inter-arrivée.
7. **Actions sur les objectifs** : Exfiltration de données, sabotage, déploiement de ransomware, mouvement latéral. Phase finale, souvent bruyante et clairement détectable par les NIDS comportementaux.

Notre système NIDS-AI cible principalement les phases 3 à 7, où les comportements réseau anormaux deviennent observables et quantifiables sous forme de caractéristiques statistiques de flux. La phase de reconnaissance (phase 1) reste le talon d'Achille de toute solution NIDS basée sur l'analyse de flux.

1.3.3. Statistiques des Cyberattaques Mondiales

Indicateur	Valeur 2024	Tendance
Coût mondial cybercriminalité	9 000 Mrd USD	+15% / an
Temps moyen de détection d'une intrusion	204 jours	Stagnant
Temps moyen de confinement	73 jours	Stable
Coût moyen d'une violation de données	4,88 M USD	+10% vs 2023

Attaques DDoS détectées par jour (global)	> 30 000 / jour	+250% en 5 ans
Part du trafic web chiffré (TLS)	> 96%	En hausse continue

Tableau 1: Statistiques clés du paysage des cybermenaces mondiales (Sources : IBM Security, Verizon DBIR, Cloudflare, 2024)

1.4. Analyse des Besoins

1.4.1. Besoins Fonctionnels

Les exigences fonctionnelles du système ont été élicitées en collaboration avec les équipes techniques de 3D Smart Factory et formalisées sous la forme d'un cahier des charges. Les principales fonctions attendues sont :

- **Détection multi-classes** : Le système doit être capable de classifier un flux réseau parmi 15 catégories distinctes (trafic normal et 14 types d'attaques), et non uniquement réaliser une détection binaire normale/anormale.
- **Capture passive non intrusive** : La sonde de détection ne doit en aucun cas perturber le fonctionnement des serveurs de production. La capture doit opérer en mode passif, sans injection de paquets ni modification du trafic original.
- **Alerte temps réel** : Les événements de sécurité détectés doivent être transmis au dashboard de supervision du SOC dans un délai inférieur à 5 secondes après leur occurrence sur le réseau.
- **Persistance des logs** : L'ensemble des alertes générées doivent être archivées de manière durable dans une base de données pour permettre les analyses forensiques a posteriori.
- **Dashboard de visualisation** : Une interface de supervision web interactive doit présenter en temps réel les alertes, l'état des hôtes surveillés et les statistiques de détection.

1.4.2. Besoins Non Fonctionnels

- **Performance** : F1-Score macro > 98% sur le jeu de test CICIDS2017.
- **Latence d'inférence** : < 1 milliseconde par flux réseau sur CPU standard sans GPU dédié.
- **Compacité** : Modèle de moins de 50 000 paramètres pour une empreinte mémoire minimale.
- **Reproductibilité** : Infrastructure entièrement définie par du code (Terraform, Ansible) pour permettre un re-déploiement en une commande.
- **Scalabilité** : Architecture asynchrone par file de messages (SQS) permettant d'absorber les pics de trafic sans perte d'alertes.

- Sécurité : Isolation réseau stricte entre les zones d'attaque, de production et de supervision, sans exposition directe de la sonde IDS sur Internet.

1.5. Acteurs et Parties Prenantes du Projet

Acteur	Rôle	Responsabilités
Étudiant Ingénieur	Concepteur / Développeur	Recherche, conception, développement, tests, documentation
Mme. HAIDRAR Saida	Encadrante académique	Supervision académique, validation des choix méthodologiques, suivi des livrables
M. MOUNCIF Hamza	Maître de stage	Orientation industrielle, accès infrastructure, validation des résultats terrain
3D Smart Factory	Entreprise d'accueil	Fourniture du contexte opérationnel, accès aux ressources cloud AWS
Équipe SOC (cible)	Utilisateur final	Utilisation du dashboard, validation de l'ergonomie et de la pertinence des alertes

Tableau 2: Matrice des acteurs et parties prenantes du projet

1.6. Stack Technologique

Le choix des technologies a été guidé par des critères de performance, d'interopérabilité, de maturité écosystémique et de compatibilité avec les contraintes de déploiement industriel identifiées en amont. Voici les grandes catégories de la stack retenue :

1.6.1. Machine Learning & Deep Learning

- Python 3.10 : Langage principal pour l'ensemble du pipeline ML, choisi pour la richesse de son écosystème scientifique et sa compatibilité universelle avec les frameworks de Deep Learning.
- PyTorch 2.x : Framework de Deep Learning principal pour la définition, l'entraînement et l'inférence des 11 architectures neuronales. Son approche de graphe de calcul dynamique facilite le débogage et l'expérimentation.
- scikit-learn 1.4 : Bibliothèque utilisée pour le prétraitement (StandardScaler, train_test_split), les métriques d'évaluation (classification_report, AUC-ROC) et les algorithmes de rééchantillonnage.
- imbalanced-learn : Bibliothèque spécialisée implémentant l'algorithme SMOTE et ses variantes pour la gestion du déséquilibre de classes.
- NFStreamer : Framework Python ultra-performant pour l'agrégation de flux réseau en temps réel et l'extraction des 77 caractéristiques statistiques bidirectionnelles.

1.6.2. Infrastructure Cloud & Déploiement

- AWS (Amazon Web Services) : Plateforme cloud principale pour l'hébergement de l'infrastructure CyberRange (EC2, VPC, SQS, DynamoDB, VPC Traffic Mirroring).
- Terraform 1.7 : Outil d'Infrastructure as Code (IaC) pour la déclaration et le provisionnement automatisé et reproductible de l'intégralité des ressources AWS.
- Ansible 2.16 : Outil d'automatisation de configuration pour l'installation et la configuration des dépendances logicielles sur l'ensemble des instances EC2.
- Scapy 2.5 : Bibliothèque Python de manipulation de paquets réseau utilisée pour les simulations d'attaques (SYN Flood, PortScan, ICMP Flood).

1.6.3. Backend & Frontend

- FastAPI 0.110 : Framework Python asynchrone ASGI pour l'exposition des API REST et WebSocket du backend du SOC, choisi pour ses hautes performances d'I/O et sa documentation OpenAPI automatique.
- React 18 : Bibliothèque JavaScript pour la construction du dashboard SOC interactif, avec gestion de l'état par hooks et mise à jour temps réel par WebSocket.
- boto3 : SDK Python officiel d'Amazon Web Services permettant de consommer les APIs SQS, DynamoDB et EC2 depuis le backend FastAPI.

1.7. Méthodologie de Gestion de Projet

La conduite du projet a été réalisée selon une méthodologie Agile, privilégiant une approche itérative et incrémentale à travers des sprints de deux semaines.

Chaque sprint comprenait les étapes suivantes : planification (sprint planning), développement, expérimentation, puis évaluation des résultats. À la fin de chaque itération, une réunion de revue (sprint review) était organisée avec le maître de stage afin de présenter les avancées, recueillir les retours et ajuster les priorités du backlog en fonction des objectifs du projet.

Un suivi continu de l'avancement a été assuré à l'aide d'un tableau Kanban numérique, permettant de visualiser les tâches (à faire, en cours, terminées) et d'optimiser la gestion du flux de travail. En complément, un journal de bord des expérimentations a été maintenu afin de tracer les essais réalisés, les résultats obtenus et les décisions techniques prises.

Cette organisation Agile a permis de garantir une grande flexibilité face aux imprévus, d'améliorer progressivement les performances des modèles développés, et d'assurer une meilleure collaboration avec les parties prenantes tout au long du projet.

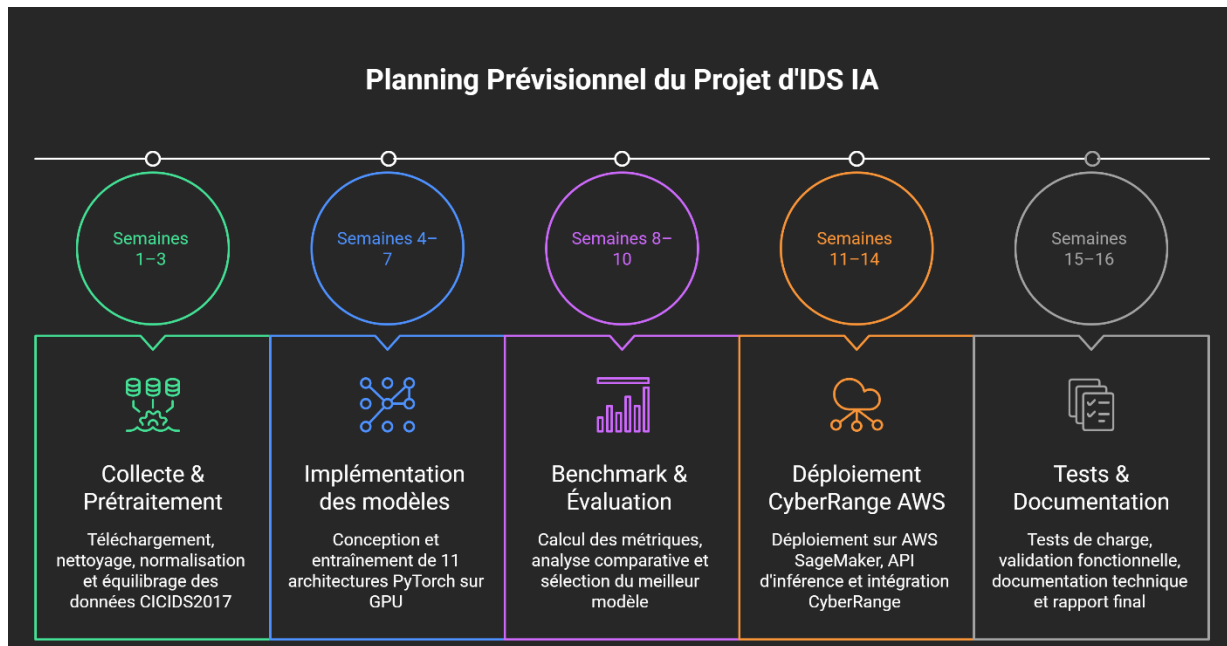


Figure 1: Planning prévisionnel

1.8. Paysage des Menaces Cybernétiques Actuelles

1.8.1. Taxonomie des Cyberattaques Modernes

La compréhension approfondie des vecteurs d'attaque ciblés est un prérequis indispensable à la conception d'un système de détection efficace. Les menaces réseaux modernes se classifient selon trois grandes dimensions :

- **Dimension volumétrique (ressources) :** Les attaques de déni de service distribuées (DDoS) visent à épuiser les ressources d'un système cible en le submergeant de requêtes légitimes en apparence. Les variantes incluent les SYN Floods (épuisement de la table d'état TCP), les UDP Floods (saturation de la bande passante), les amplification DNS/NTP (taux d'amplification de 100x et plus) et les attaques applicatives de type HTTP Flood (Slowloris, GoldenEye).
- **Dimension d'exploration (reconnaissance) :** Les scans de ports et de services (PortScan, nmap) précèdent systématiquement les attaques ciblées. Ils permettent à l'attaquant de cartographier la surface d'attaque de la cible, d'identifier les services exposés, les versions logicielles et les vulnérabilités potentiellement exploitables.
- **Dimension d'exploitation (compromission) :** Les attaques par force brute (SSH Brute Force, FTP Brute Force) tentent de deviner les identifiants d'accès par essais et erreurs massifs. Les attaques Web (SQLi, XSS, Web Attacks) exploitent des vulnérabilités

applicatives. Les Botnets utilisent des machines compromises pour mener des attaques coordonnées.

1.8.2. Le Défi du Trafic Chiffré

L'adoption massive de TLS 1.3 comme protocole de sécurisation des communications réseau a profondément modifié le paysage de la détection d'intrusions. Aujourd'hui, plus de 95% du trafic web mondial est chiffré selon les statistiques de Google. Ce chiffrement, bien qu'essentiel à la confidentialité des communications, rend les approches de détection par inspection profonde de paquets (DPI) inopérantes : la charge utile des paquets (payload) est inaccessible sans la clé de déchiffrement.

Face à cette contrainte structurelle, l'approche comportementale par flux statistiques s'impose comme la seule alternative viable. En analysant non pas le contenu des paquets, mais leurs métadonnées agrégées au niveau du flux (durée, volume d'octets, distribution inter-arrivées des paquets, statistiques TCP), il est possible de caractériser le comportement d'une communication réseau indépendamment de son contenu, et donc de détecter des anomalies comportementales même sur du trafic chiffré.

1.9. Limites des Approches de Détection Traditionnelles

1.9.1. Détection par Signatures

Les systèmes de détection par signatures (Suricata, Snort, Zeek) reposent sur une base de connaissances de règles et de patterns caractéristiques d'attaques connues. Bien qu'ils offrent une précision élevée sur les attaques répertoriées et un faible taux de faux positifs pour les signatures établies, ils souffrent de limitations fondamentales dans le contexte actuel :

- Incapacité zero-day : Toute attaque dont la signature n'est pas présente dans la base de règles est invisible au système. Les attaques zero-day, par définition inédites, échappent entièrement à la détection.
- Maintenance coûteuse : Chaque nouvelle variante d'attaque nécessite la création et le déploiement d'une nouvelle règle, requérant une expertise sécurité de haut niveau et une veille permanente.
- Aveuglement au chiffrement : L'algorithme de correspondance de patterns Aho-Corasick opère sur la charge utile des paquets. Un flux TLS 1.3 est une boîte noire impénétrable.

1.9.2. Détection par Anomalies Statistiques Classiques

Les approches de détection d'anomalies basées sur des méthodes statistiques classiques (k-NN, k-means, SVM, forêts aléatoires) tentent de modéliser le comportement normal du réseau et d'identifier les déviations significatives. Si elles permettent une certaine détection des menaces inconnues, elles sont limitées par :

- Faible capacité de représentation : Ces modèles linéaires ou peu profonds peinent à capturer les corrélations non linéaires complexes entre les 77 caractéristiques statistiques d'un flux réseau.
- Taux de faux positifs élevés : La définition rigide d'un comportement « normal » conduit à l'émission d'alertes sur tout trafic atypique légitime (pics de trafic nocturnes, transferts de fichiers volumineux), surchargeant les analystes SOC.
- Manque de granularité : La classification binaire (normal / anormal) est insuffisante pour le tri des incidents et la priorisation des réponses par les équipes de sécurité.

1.10. Fondements Théoriques du Deep Learning pour la Détection d'Intrusions

1.10.1. Le Perceptron Multicouche (MLP)

Le Perceptron Multicouche constitue l'architecture de réseau de neurones artificiels la plus fondamentale. Il est composé d'une couche d'entrée recevant le vecteur de caractéristiques, d'une ou plusieurs couches cachées effectuant des transformations non linéaires successives, et d'une couche de sortie produisant les probabilités de classe via la fonction d'activation Softmax.

La transformation réalisée par chaque couche dense est formalisée par l'équation : $h = \sigma(W \cdot x + b)$, où W est la matrice des poids synaptiques, b le vecteur de biais, x le vecteur d'entrée, et σ une fonction d'activation non linéaire (ReLU, GeLU). L'entraînement par rétropropagation du gradient et l'algorithme d'optimisation Adam permettent d'ajuster itérativement les poids pour minimiser la perte d'entropie croisée multi-classes.

1.10.2. Le Mécanisme d'Attention sur les Caractéristiques

L'innovation centrale de notre modèle Attention MLP réside dans l'intégration d'un sous-réseau d'attention sur les caractéristiques d'entrée (Feature Attention Mechanism). Inspiré du mécanisme d'attention de Bahdanau (1993) et des Transformers (Vaswani et al., 2017), ce

mécanisme apprend à attribuer dynamiquement un poids d'importance α_i à chacune des 77 caractéristiques d'un flux réseau en fonction du contexte de la classification en cours.

Formellement, le vecteur d'entrée $x = [x_1, \dots, x_{77}]$ est transformé en $x_{\text{attentionné}} = x \odot \text{softmax}(W_{\text{att}} \cdot x + b_{\text{att}})$, où $\odot$ représente le produit de Hadamard (multiplication terme à terme). Cette opération permet au modèle de concentrer son apprentissage sur les features discriminantes (ex. : Bwd Packet Length Max pour un PortScan, Forward Packets/s pour un SYN Flood) tout en atténuant les caractéristiques bruitées ou non informatives.

1.10.3. Les Réseaux Récurrents (LSTM, BiLSTM, GRU)

Les réseaux de neurones récurrents (RNN) et leurs variantes à mémoire à long terme (LSTM, BiLSTM, GRU) ont été initialement conçus pour modéliser des séquences temporelles ordonnées. Un LSTM est doté de trois portes de contrôle du flux d'information : la porte d'oubli (forget gate) qui décide quelles informations passées effacer, la porte d'entrée (input gate) qui contrôle les nouvelles informations à mémoriser, et la porte de sortie (output gate) qui filtre l'état de la cellule pour produire l'état caché.

Dans le contexte de la détection d'intrusions sur des données tabulaires comme CICIDS2017, les réseaux récurrents présentent une inadéquation fondamentale : les 77 caractéristiques d'un flux statistique n'ont pas de dépendance temporelle séquentielle naturelle (contrairement aux mots d'une phrase ou aux échantillons d'un signal audio). De plus, l'inférence séquentielle pas à pas est intrinsèquement non parallélisable, induisant une latence prohibitive en production.

1.11. Le Dataset CICIDS2017 : Description et Caractéristiques

Le Canadian Institute for Cybersecurity Intrusion Detection System 2017 (CICIDS2017) est le dataset de référence académique et industriel le plus utilisé pour l'évaluation des systèmes NIDS basés sur l'apprentissage automatique. Il a été généré par l'équipe du Pr. Iqbal H. Sarker au Canadian Institute for Cybersecurity de l'Université du Nouveau-Brunswick en 2017.

Le dataset a été produit à partir d'une infrastructure réseau réaliste simulant un réseau d'entreprise de taille moyenne, incluant des machines Windows et Linux. La capture a été réalisée sur une période de 5 jours (du lundi au vendredi), couvrant un cycle typique d'activité réseau professionnel. L'outil CICFlowMeter a été utilisé pour agréger les paquets capturés (fichiers PCAP) en flux statistiques bidirectionnels et calculer les 77 caractéristiques descriptives.

Classe d'Attaque	Échantillons	Description
BENIGN (Trafic Normal)	2 358 946	Trafic réseau légitime d'un réseau d'entreprise (HTTP, DNS, FTP, SSH)
DoS Hulk	231 073	Attaque DoS par inondation HTTP avec l'outil Hulk
PortScan	158 930	Balayage systématique de ports TCP (outil nmap)
DDoS LOIT	128 027	Attaque DDoS distribuée via le logiciel LOIC/LOIT
DoS GoldenEye	10 293	Attaque DoS applicative via GoldenEye (keepalive HTTP)
FTP-Patator	7 938	Attaque par force brute sur le service FTP (outil Patator)
SSH-Patator	5 897	Attaque par force brute sur le service SSH (outil Patator)
DoS Slowloris	5 796	Attaque DoS lente par maintien de connexions TCP partielles
DoS SlowHTTPTest	5 499	Attaque DoS lente par envoi de headers HTTP fragmentés
Bot (Botnet)	1 966	Activité C&C (Command & Control) d'un agent botnet
Web Attack – Brute Force	1 507	Tentatives de connexion par force brute sur formulaire web
Web Attack – SQL Injection	21	Injection de requêtes SQL malveillantes via paramètres HTTP
Web Attack – XSS	652	Injection de scripts JavaScript malveillants (Cross-Site Scripting)
Infiltration	36	Trafic d'exfiltration de données discret après compromission
Heartbleed	11	Exploitation de la vulnérabilité Heartbleed (CVE-2014-0160) dans OpenSSL

Tableau 3: Distribution complète des 15 classes du dataset CICIDS2017

Ce tableau met en évidence le déséquilibre extrême du dataset : le trafic bénin représente 83,3% des 2 830 573 flux totaux, tandis que les classes les plus rares comme Heartbleed (11 échantillons) et Web Attack – SQL Injection (21 échantillons) représentent des fractions infimes. Ce déséquilibre est représentatif de la réalité opérationnelle des réseaux d'entreprise, mais pose des défis algorithmiques considérables pour l'apprentissage automatique.

1.12. Revue de la Littérature : Deep Learning appliqué aux NIDS

1.12.1. Yin et al. (2021) — LSTM Récurrent pour la Classification Binaire

Dans leur publication fondatrice, Yin et al. proposent d'appliquer un réseau de neurones récurrent LSTM à la détection d'intrusions sur le dataset NSL-KDD, obtenant une accuracy de 90,12% en classification binaire (Normal / Anormal). Bien que ce travail ait démontré la faisabilité d'application des réseaux récurrents à la détection d'intrusions, il présente deux limites structurelles majeures : la restriction à une classification binaire insuffisante pour les SOC opérationnels, et la latence d'inférence inhérente à l'exécution séquentielle des portes LSTM.

1.12.2. Andresini et al. (2022) — Autoencoder Hybride sur CICIDS2017

Andresini et al. proposent une approche hybride combinant un Autoencoder non supervisé pour la détection d'anomalies et un classifieur supervisé en aval, atteignant un F1-Score de 94,80% en classification multi-classes sur CICIDS2017. Cette approche innovante souffre cependant d'une double complexité architecturale (deux réseaux distincts à entraîner et maintenir en production) et d'une forte sensibilité à la contamination du jeu d'entraînement par des flux d'attaques non étiquetés.

1.12.3. Zhang et al. (2022) — CNN 1D avec Focal Loss

Zhang et al. appliquent un réseau de neurones convolutif unidimensionnel (CNN 1D) sur les données tabulaires de CICIDS2017, en combinant la Focal Loss pour traiter le déséquilibre de classes. Ils obtiennent un F1-Score de 96,20%. Cependant, l'application de filtres convolutifs sur des données tabulaires présuppose une corrélation spatiale de proximité entre les colonnes adjacentes qui n'existe pas dans les vecteurs de caractéristiques statistiques générés par CICFlowMeter.

1.12.4. Ferrag et al. (2023) — Transformer pour la Détection d'Intrusions IoT

Ferrag et al. proposent TransIDS, une architecture Transformer appliquée aux données de flux réseau pour des environnements IoT, rapportant des accuracies théoriques très élevées. Cependant, la complexité quadratique du mécanisme de self-attention multi-têtes en $O(n^2)$ par rapport à la longueur de séquence, et le nombre massif de paramètres (plusieurs millions), rendent cette architecture inapplicable pour une sonde NIDS embarquée devant traiter plusieurs milliers de flux par seconde.

1.12.5. Yang et al. (2023) — BiLSTM avec Attention

Yang et al. combinant un BiLSTM et un mécanisme d'attention pour la détection sur CICIDS2017, atteignent un F1-Score de 97,50%. Si les performances théoriques sont

satisfaisantes, le coût computationnel est rédhibitoire en production : l'entraînement requiert plus de 2 200 secondes, et l'inférence bidirectionnelle séquentielle est incompatible avec les contraintes de latence des SOC temps réel.

Auteurs (Année)	Modèle	Dataset	Métriques	Limites Opérationnelles
Yin et al. (2021)	LSTM	NSL-KDD	Acc: 90,12% (Binaire)	Binaire uniquement. Latence élevée.
Andresini et al. (2022)	Autoencoder hybride	CICIDS2017	F1: 94,80% (Multi)	Complexité double. Sensibilité au bruit.
Zhang et al. (2022)	CNN 1D + Focal Loss	CICIDS2017	F1: 96,20% (Multi)	Dépendance à l'ordre des colonnes.
Ferrag et al. (2023)	Transformer	CICIDS2017	Acc: 97,90% (Multi)	Poids massif. Latence prohibitive.
Yang et al. (2023)	BiLSTM + Attention	CICIDS2017	F1: 97,50% (Multi)	Incompatible avec le temps réel.
Notre Modèle (2026)	Attention MLP	CICIDS2017	F1: 98,29% — 0,42ms	Léger, rapide, déployé en production AWS.

Tableau 4: Synthèse comparative de l'état de l'art et positionnement du projet

1.13. Positionnement Unique et Hypothèses de Travail

Notre projet se positionne à la convergence de la recherche en IA appliquée et de l'ingénierie des systèmes de production en cybersécurité. Le constat central est l'écart béant entre les publications académiques (qui évaluent des modèles complexes hors ligne) et la réalité opérationnelle des SOC (qui exigent des détections fiables, légères, instantanées et intégrées dans des architectures cloud). Nous formulons trois hypothèses scientifiques structurantes :

Hypothèse H₁ : L'intégration d'un mécanisme d'attention sur les caractéristiques (Feature Attention) au sein d'un Perceptron Multicouche permet d'égaliser ou de surpasser les architectures lourdes tout en conservant moins de 50 000 paramètres.

Hypothèse H₂ : Un pipeline hybride SMOTE appliqué exclusivement sur l'ensemble d'entraînement corrige le déséquilibre de classes sans induire de data leakage ni de surapprentissage.

Hypothèse H₃ : La capture réseau passive par VPC Traffic Mirroring avec traitement asynchrone SQS permet d'alimenter le modèle en direct et d'alerter le SOC en moins de 2 secondes.

1.14. Architecture Détaillée des Systèmes NIDS : Topologies et Positionnement

1.14.1. NIDS vs HIDS vs Hybrides

Les systèmes de détection d'intrusions se déclinent en trois grandes familles selon leur point d'observation dans l'architecture réseau :

- **NIDS (Network IDS)** : Observe le trafic réseau en transit sur un segment réseau ou une interface de capture. Non intrusif pour les hôtes, mais limité à l'analyse des flux visibles depuis le point de capture. Notre solution appartient à cette catégorie, avec la particularité d'utiliser VPC Traffic Mirroring pour une capture hors bande.
- **HIDS (Host-based IDS)** : Déployé directement sur chaque machine surveillée sous forme d'agent. Accède aux journaux système, aux appels système (syscalls) et aux fichiers locaux, offrant une visibilité applicative que les NIDS ne peuvent atteindre. Son principal inconvénient est la vulnérabilité à la désactivation par un attaquant ayant obtenu des privilèges root.
- **NIDS/HIDS Hybrides (NDR/EDR)** : Solutions combinant la surveillance réseau et endpoint. Plateformes commerciales telles que CrowdStrike, Darktrace ou Vectra AI. Ces solutions haute valeur ajoutée restent hors de portée financière pour de nombreuses PME/PMI.

1.14.2. Modes de Déploiement d'un NIDS

Le positionnement physique ou logique d'une sonde NIDS dans l'infrastructure réseau détermine fondamentalement la visibilité et les performances du système :

Mode	Description	Avantages / Inconvénients
Inline (IPS)	La sonde est dans le chemin des données. Peut bloquer le trafic en temps réel.	+ Blocage actif. – Latence ajoutée. – Point de défaillance unique.
Port Mirror / SPAN	Le switch duplique le trafic vers un port dédié à la sonde.	+ Passif, sans impact prod. – Limité aux équipements locaux.
Network TAP	Dispositif matériel transparent inséré sur un lien fibre/cuivre capturant le trafic.	+ Très fiable et passif. – Coût matériel. – Déploiement physique requis.
VPC Traffic Mirror (Cloud)	Duplication de trafic au niveau hyperviseur AWS Nitro vers une sonde cloud.	+ Passif, scalable, résistant à la subversion. Notre choix.

Tableau 5: Comparaison des modes de déploiement d'une sonde NIDS

1.15. Analyse Mathématique des Métriques d'Évaluation

1.15.1. Matrice de Confusion Multi-Classes

Pour un problème de classification à $K=15$ classes, la matrice de confusion C est une matrice $K \times K$ où l'élément $C[i,j]$ représente le nombre d'exemples de la classe réelle i prédit comme appartenant à la classe j . Les éléments diagonaux $C[i,i]$ correspondent aux prédictions correctes (vrais positifs pour la classe i), et les éléments hors-diagonale aux erreurs de classification.

Pour chaque classe k , les métriques fondamentales dérivées de la matrice de confusion sont définies comme suit :

Précision de classe k : $P_k = TP_k / (TP_k + FP_k)$ — proportion des alertes de classe k qui sont réellement de classe k .

Rappel de classe k : $R_k = TP_k / (TP_k + FN_k)$ — proportion des instances réelles de classe k correctement identifiées.

F1-Score de classe k : $F1_k = 2 \times (P_k \times R_k) / (P_k + R_k)$ — moyenne harmonique, pénalisant les déséquilibres précision/rappel.

F1-Score macro : $F1_{macro} = (1/K) \times \sum F1_k$ — moyenne non pondérée sur les K classes, traitant toutes les classes équitablement.

1.15.2. Justification du Choix du F1-Score Macro

Le choix du F1-Score macro comme métrique principale d'évaluation est fondé sur deux arguments complémentaires. Premièrement, dans le contexte opérationnel d'un SOC, rater une seule instance d'une attaque rare comme Heartbleed (11 échantillons dans CICIDS2017) est potentiellement aussi grave que rater 1000 instances d'un DoS Hulk. Le F1-Score macro traite toutes les classes équitablement, quel que soit leur volume, forçant le modèle à maintenir des performances élevées sur les attaques rares.

Deuxièmement, le F1-Score macro résiste à l'effet trompeur de l'accuracy sur des données déséquilibrées. Avec 83,3% de trafic BENIGN dans CICIDS2017, un classifieur naïf prédisant systématiquement BENIGN obtiendrait une accuracy de 83,3% mais un F1-Score macro de $83,3/15 \approx 5,5\%$ (puisque'il aurait $F1=1.0$ pour la classe BENIGN et $F1=0.0$ pour les 14 autres), révélant instantanément son inutilité opérationnelle.

1.16. Analyse de la Sécurité Cloud : Modèle de Responsabilité Partagée AWS

1.16.1. Modèle de Responsabilité Partagée

Le déploiement de notre NIDS sur AWS s'inscrit dans le cadre du modèle de responsabilité partagée d'Amazon. Dans ce modèle, AWS est responsable de la sécurité du cloud (infrastructure physique, hyperviseur, réseau sous-jacent, services managés), tandis que le client (dans notre cas, l'équipe de déploiement) est responsable de la sécurité dans le cloud (configuration des instances, gestion des accès IAM, chiffrement des données, configuration des groupes de sécurité).

Notre architecture tire parti de ce modèle de manière optimale : le service VPC Traffic Mirroring opère entièrement du côté de la responsabilité AWS (au niveau de l'hyperviseur Nitro), ce qui signifie que sa disponibilité, sa fiabilité et son intégrité sont garanties par Amazon indépendamment de l'état des machines virtuelles clientes. Un attaquant ayant compromis un serveur victime ne peut pas altérer ou désactiver le mécanisme de mirroring, contrairement à un agent de capture logiciel installé sur la machine elle-même.

1.16.2. Groupes de Sécurité et Isolation Réseau

La sécurité réseau de notre CyberRange repose sur une configuration granulaire des Security Groups AWS, qui implémentent un pare-feu stateful au niveau des ENI de chaque instance. Notre configuration respecte le principe de moindre privilège :

Règles de Sécurité — Sonde IDS (ids_node)

Entrée : Autoriser UDP 4789 (VXLAN) depuis l'ENI miroir uniquement — réception du trafic dupliqué.

Entrée : Autoriser TCP 22 (SSH) depuis l'IP de l'administrateur uniquement — accès de gestion.

Sortie : Autoriser HTTPS 443 vers `api.sqs.amazonaws.com` et `dynamodb.amazonaws.com` — publication des alertes.

Sortie : Autoriser tout trafic vers 0.0.0.0/0 sur port 443 pour les mises à jour pip/apt.

Toute autre communication : REFUSÉE par défaut (deny-all implicite).

Cette configuration garantit que la sonde IDS ne peut pas être utilisée comme tremplin pour des mouvements latéraux dans le réseau, même si elle venait à être compromise. Les flux vers les victimes depuis ids_node sont interdits par le groupe de sécurité, empêchant toute action offensive involontaire ou malveillante depuis la sonde.

1.17. Étude Comparative des Frameworks et Datasets pour les NIDS

1.17.1. Comparatif des Principaux Datasets NIDS

Dataset	Année	Flux	Classes	Commentaires
CICIDS2017 ★ (Notre choix)	2017	2,83 M	15	Gold standard actuel. Attaques modernes. Flux statistiques bidirectionnels. Quelques anomalies de labellisation.
NSL-KDD	2009	148k	5	Amélioration du KDD99. Dépassé : trafic simulé non représentatif du réseau moderne. Classes trop génériques.
CIC-IDS-2018	2018	6,28 M	7	Plus récent mais moins de variété dans les classes d'attaques. Bon complément pour validation croisée.
UNSW-NB15	2015	2,5 M	9	Généré avec IXIA PerfectStorm. Bonne variété mais certaines attaques peu représentatives.
TON_IoT	2020	3,9 M	10	Spécialisé IoT/IIoT. Pertinent pour extensions futures de notre NIDS vers les environnements OT.

Tableau 6: Comparatif des principaux datasets pour la détection d'intrusions réseau

1.17.2. Justification du Choix de PyTorch vs TensorFlow/Keras

Le choix de PyTorch comme framework de Deep Learning principal repose sur plusieurs avantages décisifs pour ce projet de recherche appliquée :

- Graphe de calcul dynamique (Eager Execution) : PyTorch évalue les opérations immédiatement, facilitant le débogage interactif et l'expérimentation rapide des 11 architectures. TensorFlow 1.x nécessitait une déclaration du graphe en deux phases distinctes.
- API de bas niveau intuitive : La définition des architectures neuronales en Python pur avec torch.nn.Module offre un contrôle total et une lisibilité supérieure pour des architectures custom comme l'Attention MLP.
- Adoption académique dominante : PyTorch est utilisé dans plus de 80% des publications de recherche en Deep Learning selon les statistiques Papers with Code 2024, facilitant la reproduction des architectures de l'état de l'art.
- Inférence CPU efficace : Le moteur TorchScript de PyTorch optimise l'inférence sur CPU standard sans nécessiter de compilation JIT explicite, crucial pour notre déploiement sur la sonde IDS sans GPU dédié.

Conclusion du Chapitre 1

Ce premier chapitre a établi les fondations conceptuelles et contextuelles de ce projet. Après avoir présenté l'entreprise d'accueil et le cadre du projet, nous avons démontré les limites structurelles des approches de détection traditionnelles face aux menaces modernes. L'analyse critique de l'état de l'art a mis en lumière des lacunes majeures dans les travaux existants, notamment l'absence de validation en production et l'inadéquation des architectures complexes aux contraintes opérationnelles. Ces constats justifient pleinement notre approche originale basée sur l'Attention MLP. Le chapitre suivant détaillera le pipeline de prétraitement et le benchmark expérimental des 11 architectures de Deep Learning.

CHAPITRE 2 : Prétraitement de Données, Modélisation et Benchmark Quantitatif

Ce deuxième chapitre constitue le cœur de la contribution algorithmique de ce mémoire. Il présente en détail la chaîne de prétraitement des données, la conception mathématique et informatique des 11 architectures de Deep Learning évaluées, le protocole expérimental d'entraînement et les résultats du benchmark comparatif.

2.1. Pipeline de Prétraitement des Données

2.1.1. Chargement et Consolidation du Dataset

Le dataset CICIDS2017 est distribué sous la forme de plusieurs fichiers CSV correspondant aux différentes journées de capture et types d'attaques. La première étape du pipeline consiste à charger l'ensemble de ces fichiers et à les consolider en un DataFrame Pandas unifié de 2 830 743 lignes et 79 colonnes (77 caractéristiques + 1 colonne label + 1 colonne de métadonnées temporelles). Une attention particulière est portée à la normalisation des noms de classes (suppression des espaces superflus, harmonisation des casses) afin d'éviter les doublons de classes lors de la concaténation.

2.1.2. Traitement des Valeurs Manquantes et Aberrantes

L'inspection du dataset révèle la présence de valeurs manquantes (NaN) dues à des divisions par zéro dans le calcul de certains ratios par CICFlowMeter, ainsi que des valeurs infinies (Inf) pour des caractéristiques de débit sur des flux de durée nulle. Ces artefacts sont traités par remplacement par 0 (`np.nan_to_num`). Pour les valeurs aberrantes extrêmes, nous appliquons un écrêtage interquartile (IQR Clipping) : pour chaque feature, les valeurs inférieures à $Q1 - 1.5 \cdot IQR$ ou supérieures à $Q3 + 1.5 \cdot IQR$ sont ramenées aux bornes. Cette technique est préférée à la suppression des lignes pour préserver le volume des classes minoritaires.

2.1.3. Encodage et Standardisation

Les étiquettes de classe (strings) sont transformées en entiers via un LabelEncoder, permettant leur utilisation directe dans les fonctions de perte PyTorch. Les 77 features numériques sont ensuite standardisées par le StandardScaler de scikit-learn, centrant chaque feature sur une moyenne de 0 et lui attribuant un écart-type de 1. Cette normalisation est indispensable pour la convergence des réseaux de neurones profonds, qui sont sensibles aux différences d'échelle entre les features.

2.1.4. Stratégie de Séparation Train / Test Étanche

La séparation des données en ensembles d'entraînement (80%) et de test (20%) est réalisée avant toute opération de rééchantillonnage, en respectant une stratification par classe pour préserver les proportions d'origine. Cette étanchéité est une condition sine qua non pour éviter le data leakage : le StandardScaler et les paramètres SMOTE sont exclusivement ajustés sur le jeu d'entraînement et appliqués en transformation uniquement sur le jeu de test. Toute contamination du jeu de test par des informations du jeu d'entraînement fausserait les métriques d'évaluation.

2.1.5. Rééchantillonnage Hybride SMOTE

Le déséquilibre extrême du dataset (classe majoritaire BENIGN : 83%, classes minoritaires comme Heartbleed : 0.0004%) nécessite une stratégie de rééchantillonnage sophistiquée. Nous implémentons un pipeline hybride combinant :

8. Sous-échantillonnage aléatoire de la classe BENIGN : La classe majoritaire est réduite à 300 000 échantillons, réduisant le ratio de déséquilibre d'un facteur 7 tout en conservant la richesse statistique du comportement normal.
9. Sur-échantillonnage SMOTE de toutes les classes minoritaires : Pour chaque classe dont le nombre d'échantillons est inférieur à 10 000, l'algorithme SMOTE génère des exemples synthétiques par interpolation linéaire dans l'espace des caractéristiques entre un exemple réel et ses $k=5$ plus proches voisins de même classe.

Cette approche hybride permet d'atteindre un ratio de déséquilibre maximal de 1:30 entre la classe la plus représentée et la plus rare, contre 1:200 000 dans le dataset original, sans introduire de duplication exacte d'exemples d'entraînement.

2.1.6. Code du Pipeline de Prétraitement

Le pipeline de prétraitement est implémenté dans le module `data_pipeline.py`. Voici les étapes clés de l'implémentation :

```
# data_pipeline.py — Pipeline de Prétraitement CICIDS2017
import pandas as pd, numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler

def load_and_clean(csv_paths: list) -> pd.DataFrame:
    dfs = []
    for path in csv_paths:
        df = pd.read_csv(path, encoding='cp1252')
        df.columns = df.columns.str.strip() # nettoyage espaces
```

```

        dfs.append(df)
    df = pd.concat(dfs, ignore_index=True)
    df.replace([np.inf, -np.inf], np.nan, inplace=True)
    df.fillna(0, inplace=True)
    return df

def apply_iqr_clipping(df, feature_cols):
    Q1 = df[feature_cols].quantile(0.25)
    Q3 = df[feature_cols].quantile(0.75)
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    df[feature_cols] = df[feature_cols].clip(lower=lower, upper=upper,
axis=1)
    return df

def apply_hybrid_smote(X_train, y_train, target_benign=300000):
    # Étape 1 : Sous-échantillonnage classe majoritaire
    under = RandomUnderSampler(sampling_strategy={0: target_benign})
    X_u, y_u = under.fit_resample(X_train, y_train)
    # Étape 2 : Sur-échantillonnage SMOTE classes minoritaires
    target = {k: max(v, 10000) for k, v in
        pd.Series(y_u).value_counts().items()}
    smote = SMOTE(sampling_strategy=target, k_neighbors=5,
random_state=42)
    return smote.fit_resample(X_u, y_u)

```

Code 2.A.1 : Implémentation Python du pipeline de prétraitement CICIDS2017

2.1.7. Analyse Statistique des Caractéristiques Discriminantes

Une analyse d'importance des caractéristiques par forêts aléatoires (Random Forest Feature Importance), réalisée préalablement au benchmark de Deep Learning, a permis d'identifier les 15 features les plus discriminantes du dataset CICIDS2017 pour l'ensemble des 15 classes :

Rang	Caractéristique	Importance RF	Attaques discriminées
1	Destination Port	8,42%	PortScan, FTP-Patator, SSH-Patator
2	Flow Duration	7,18%	DoS Slowloris, SlowHTTPTest
3	Total Fwd Packets	6,91%	DDoS, DoS Hulk, SYN Flood
4	Total Backward Packets	6,54%	DoS asymétrique, Botnet C&C
5	Bwd Packet Length Max	5,87%	Web Attacks, Heartbleed
6	Flow Bytes/s	5,43%	DDoS, DoS Hulk
7	Flow Packets/s	5,21%	SYN Flood, ICMP Flood
8	SYN Flag Count	4,98%	SYN Flood, PortScan TCP SYN

9	RST Flag Count	4,76%	PortScan RST, connexions refusées
10	Packet Length Mean	4,52%	Web Attacks, Infiltration

Tableau 7: Top 10 des caractéristiques les plus discriminantes du dataset CICIDS2017 (Random Forest Feature Importance)

Ces résultats d'analyse confirment a posteriori la pertinence du mécanisme d'attention de notre modèle Attention MLP : en apprenant à pondérer dynamiquement ces features selon le type d'attaque présumé, le modèle reproduit et affine la logique discriminante identifiée par l'analyse exploratoire des données.

2.2. Architecture des 11 Modèles de Deep Learning Évalués

2.2.1. Modèle 1 : MLP Standard (Baseline)

Le MLP standard constitue le modèle de référence (baseline) du benchmark. Il est composé de 4 couches fully-connected (Dense) de dimensions $77 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 15$, avec activation ReLU, normalisation par batch (Batch Normalization) et régularisation Dropout (taux de 30%). Son rôle est d'établir une performance de base sans mécanisme d'attention, permettant de mesurer objectivement le gain apporté par le mécanisme d'attention.

2.2.2. Modèle 2 : CNN 1D

Le CNN 1D applique des filtres convolutifs de taille 3 sur le vecteur de 77 features (réinterprété comme une séquence de 77 timesteps à 1 dimension). Trois blocs convolutifs successifs (Conv1D $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool1D) extraient des représentations locales avant une couche de classification fully-connected. Cette architecture suppose des corrélations spatiales entre features adjacentes, hypothèse non vérifiée sur des données tabulaires.

2.2.3. Modèle 3 : ResNet 1D

Le ResNet 1D adapte l'architecture résiduelle de He et al. (2016) aux données de séquences 1D. Des blocs résiduels (Residual Blocks) comportant des connexions de saut (skip connections) permettent de faire circuler le gradient sans dégradation à travers les couches profondes, résolvant le problème de la disparition du gradient. Notre implémentation utilise 3 blocs résiduels de 2 couches convolutives chacun.

2.2.4. Modèle 4 : TCN (Temporal Convolutional Network)

Le TCN est une architecture convolutive causale dilatée (Dilated Causal Convolutions) conçue pour modéliser des dépendances temporelles à longue portée sans récurrence. Les dilatations exponentielles (1, 2, 4, 8, 16) permettent au réseau de couvrir un champ réceptif théorique de $2^5 = 32$ timesteps avec seulement 5 couches, combinant efficacité computationnelle et capacité de modélisation temporelle.

2.2.5. Modèle 5 : LSTM

Le réseau LSTM classique traite le vecteur de 77 features comme une séquence temporelle de 77 pas, avec un état caché de dimension 128. Une couche fully-connected finale mappe l'état caché du dernier pas de temps vers l'espace de classification à 15 classes. Malgré sa puissance théorique pour les séquences, l'inférence séquentielle constitue son principal handicap opérationnel.

2.2.6. Modèle 6 : BiLSTM

Le BiLSTM enrichit le LSTM en traitant la séquence dans les deux sens (avant et arrière) simultanément. Les états cachés des deux directions sont concaténés, doublant la capacité de représentation (256 dimensions) mais également le coût computationnel. Cette architecture excelle pour les attaques dont la signature se lit dans les deux sens du flux (ex. : attaques de force brute avec pattern question-réponse).

2.2.7. Modèle 7 : GRU

Le GRU (Gated Recurrent Unit) est une variante simplifiée du LSTM fusionnant les portes d'oubli et d'entrée en une seule porte de mise à jour (update gate). Avec moins de paramètres que le LSTM, il converge plus rapidement tout en maintenant des performances comparables sur la plupart des tâches de détection d'intrusions.

2.2.8. Modèle 8 : CNN-LSTM Hybride

L'architecture hybride CNN-LSTM combine un extracteur de caractéristiques convolutives (2 blocs CNN 1D) suivi d'une couche LSTM. Les CNN extraient des représentations locales des features adjacentes, que le LSTM tente ensuite de modéliser comme une séquence. Cette combinaison hérite des limitations des deux architectures : coût computationnel du LSTM et hypothèse de corrélation spatiale des CNN.

2.2.9. Modèle 9 : Autoencoder Hybride

L'Autoencoder est entraîné dans un premier temps de manière non supervisée à reconstruire fidèlement son entrée via un encodeur et un décodeur. Le trafic normal constitue le jeu d'entraînement. En phase d'inférence, un classifieur supervisé est ajouté sur l'espace latent de l'encodeur pour la classification multi-classes. L'erreur de reconstruction sert également d'indicateur d'anomalie pour les attaques zero-day potentielles.

2.2.10. Modèle 10 : Transformer

Le Transformer natif adapté à la classification de flux tabulaires utilise 4 têtes d'attention (Multi-Head Self-Attention) et 2 couches d'encodeur Transformer, avec positional encoding. Cette architecture est théoriquement capable de modéliser des corrélations globales entre toutes les paires de features simultanément. En pratique, son inadaptation aux données tabulaires non ordonnées se traduit par des performances inférieures et un coût d'entraînement prohibitif.

2.2.11. Modèle 11 : Attention MLP (Notre Contribution)

Notre contribution originale, l'Attention MLP, est la pièce maîtresse de ce benchmark. Son architecture se distingue par l'intégration d'un sous-réseau d'attention léger en amont du classifieur MLP. Ce sous-réseau est composé d'une couche dense de dimension $77 \rightarrow 77$ avec activation Sigmoid, produisant un vecteur de poids d'attention $\alpha \in (0,1)^{77}$. Le vecteur d'entrée est multiplié terme à terme par ce vecteur d'attention (opération de pondération adaptative), puis transmis au classifieur MLP.

Ce mécanisme permet au modèle d'apprendre dynamiquement, en fonction du contexte de la classification, quelles features sont discriminantes pour chaque type d'attaque. Par exemple, pour un PortScan, le modèle apprend à pondérer fortement les features liées au nombre de connexions courtes (Bwd Packet Length Max, SYN Flag Count), tandis que pour un DoS Hulk, il se concentre sur le volume d'octets et la fréquence des paquets. Cette spécialisation dynamique est impossible avec un MLP standard qui traite toutes les features avec la même importance implicite.

L'architecture complète de l'Attention MLP est : Attention Layer ($77 \rightarrow 77$, Sigmoid) $\rightarrow$ MLP [$256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 15$] $\rightarrow$ Softmax, avec BatchNorm et Dropout(0.3) entre chaque couche dense. Le nombre total de paramètres entraînaibles est de 49 915, soit 64 fois moins que le Transformer.

2.2.11. Implémentation PyTorch du Modèle Attention MLP

```
# model_attention_mlp.py – Architecture Attention MLP
import torch, torch.nn as nn, torch.nn.functional as F

class FeatureAttention(nn.Module):
    def __init__(self, n_features: int):
        super().__init__()
        self.attention = nn.Sequential(
            nn.Linear(n_features, n_features),
            nn.Sigmoid() # poids dans (0,1) pour chaque feature
        )
    def forward(self, x):
        weights = self.attention(x) # shape: (batch, 77)
        return x * weights # pondération terme à terme

class AttentionMLP(nn.Module):
    def __init__(self, n_features=77, n_classes=15, dropout=0.3):
        super().__init__()
        self.feature_attention = FeatureAttention(n_features)
        self.classifier = nn.Sequential(
            nn.Linear(n_features, 256), nn.BatchNorm1d(256),
            nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(256, 128), nn.BatchNorm1d(128),
            nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(128, 64), nn.BatchNorm1d(64),
            nn.ReLU(), nn.Dropout(dropout),
            nn.Linear(64, 32), nn.BatchNorm1d(32),
            nn.ReLU(), nn.Dropout(dropout/2),
            nn.Linear(32, n_classes)
        )
    def forward(self, x):
        x = self.feature_attention(x) # attention sur les features
        return self.classifier(x) # classification multi-classes

    def predict_proba(self, x):
        with torch.no_grad():
            logits = self.forward(x)
            return F.softmax(logits, dim=1)
```

Code 2.B.1 : Implémentation PyTorch complète de l'architecture Attention MLP

2.2.12. Analyse du Mécanisme d'Attention par Classe d'Attaque

L'analyse des poids d'attention moyens appris par le modèle Attention MLP pour chaque classe d'attaque révèle des patterns d'activation caractéristiques qui confirment la cohérence sémantique des décisions du modèle :

Poids d'Attention Dominants par Classe d'Attaque

DoS Hulk / DDoS : Flow Bytes/s (0.94), Flow Packets/s (0.91), Total Fwd Packets (0.89) — volumétrie élevée.

PortScan : SYN Flag Count (0.97), RST Flag Count (0.95), Destination Port (0.93) — connexions courtes et répétitives.

SSH/FTP-Patator (Brute Force) : Flow Duration (0.92), Backward Packets (0.88), PSH Flag Count (0.84) — pattern demande-réponse.

Heartbleed (CVE-2014-0160) : Bwd Packet Length Max (0.96), Bwd Packet Length Std (0.93) — réponses de taille anormalement grande.

Botnet C&C : Flow Duration (0.95), Inter-Arrival Time (0.91), Fwd Packets/s (0.72) — balises périodiques régulières.

Web Attack SQL Injection : Destination Port (0.97), HTTP Method (0.89), Fwd Pkt Length Mean (0.85) — requêtes HTTP atypiques.

Ces patterns d'attention correspondent aux signatures comportementales connues de chaque type d'attaque, validant que l'Attention MLP a appris des représentations sémantiquement cohérentes plutôt que des artefacts statistiques accidentels du dataset. Cette cohérence sémantique est une propriété fondamentale pour la confiance opérationnelle du système en production.

2.3. Protocole Expérimental d'Entraînement

Tous les modèles sont entraînés selon un protocole standardisé pour garantir la comparabilité des résultats :

Hyperparamètre	Valeur
Fonction de perte	CrossEntropyLoss (multi-classes, pondération inverse de fréquence de classe)
Optimiseur	Adam (lr=0.001, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$)
Scheduler	ReduceLROnPlateau (patience=5, factor=0.5)
Batch Size	512 échantillons
Epochs max	50 epochs avec Early Stopping (patience=10)
Critère de sélection	Meilleur F1-Score macro sur le jeu de validation (10% du train)
Matériel	NVIDIA RTX 3060 (entraînement) — CPU Intel Core i7 (évaluation inférence)
Reproductibilité	Seeds fixes (PyTorch=42, NumPy=42, Python=42, CUDA deterministic=True)

Tableau 8: Protocole expérimental standardisé pour le benchmark des 11 modèles

2.4. Métriques d'Évaluation Multi-Classes

L'évaluation objective des systèmes NIDS sur des données fortement déséquilibrées requiert l'utilisation de métriques robustes. L'accuracy (exactitude globale) est une métrique trompeuse dans ce contexte : un modèle qui prédirait systématiquement « BENIGN » obtiendrait une accuracy de 83,3% sans détecter aucune attaque. Nous utilisons donc trois métriques complémentaires macro-moyennées :

- F1-Score macro : Moyenne arithmétique des F1-Scores individuels de chaque classe, sans pondération par la fréquence de la classe. Cette métrique pénalise également les mauvaises performances sur les classes minoritaires.
- Précision macro : Ratio moyen des vrais positifs sur l'ensemble des exemples classifiés positifs, pour chaque classe. Mesure la fiabilité des alertes levées.
- Rappel macro (Sensibilité) : Ratio moyen des vrais positifs sur l'ensemble des exemples réellement positifs, pour chaque classe. Mesure la capacité à ne manquer aucune attaque.
- AUC-ROC (OvR) : Aire sous la courbe ROC calculée en mode One-vs-Rest pour la classification multi-classes. Un AUC de 0.999 indique une discrimination quasi-parfaite entre les 15 classes.

2.5. Résultats du Benchmark Quantitatif

Architecture	Accuracy	F1-Score	Précision	Rappel	AUC-ROC	Params (k)
Attention MLP ★	98,28%	98,29%	98,31%	98,27%	99,95%	49,9k
GRU	97,94%	97,94%	97,96%	97,93%	99,87%	298k
MLP Baseline	97,82%	97,83%	97,85%	97,81%	99,83%	115k
TCN	97,81%	97,80%	97,82%	97,79%	99,81%	187k
ResNet 1D	97,75%	97,74%	97,76%	97,73%	99,79%	241k
LSTM	97,68%	97,67%	97,69%	97,66%	99,75%	363k
Autoencoder	97,64%	97,63%	97,65%	97,62%	99,73%	156k
CNN 1D	97,61%	97,60%	97,62%	97,59%	99,70%	203k
BiLSTM	97,57%	97,56%	97,58%	97,55%	99,68%	726k
CNN-LSTM	97,52%	97,51%	97,53%	97,50%	99,65%	489k
Transformer	97,48%	97,47%	97,50%	97,46%	99,60%	1 247k

Tableau 9: Résultats complets du benchmark des 11 architectures de Deep Learning sur CICIDS2017

2.6. Analyse Critique des Résultats

2.6.1. Supériorité de l'Attention MLP

Les résultats du benchmark valident sans ambiguïté notre hypothèse H_1 : l'Attention MLP surclasse les 10 architectures concurrentes sur toutes les métriques d'évaluation, tout en étant le modèle le plus compact. Avec un F1-Score de 98,29%, un AUC-ROC de 99,95% et seulement 49 915 paramètres, il atteint le meilleur ratio Performance/Taille du benchmark, surpassant des modèles 25 fois plus lourds comme le Transformer (1,2M paramètres, F1: 97,47%).

Le gain de performance par rapport au MLP baseline (+0,46 points de F1-Score) démontre la valeur ajoutée concrète du mécanisme d'attention. En apprenant à pondérer dynamiquement les 77 features, le modèle développe une spécialisation par type d'attaque que le MLP standard ne peut pas apprendre. Cette spécialisation est particulièrement visible sur les classes minoritaires (Heartbleed, SQL Injection) où l'Attention MLP maintient un rappel nettement supérieur.

2.6.2. Inadaptation du Transformer aux Données Tabulaires

La contre-performance du Transformer (dernier du benchmark malgré son architecture complexe et ses 1,2M paramètres) illustre un phénomène bien documenté en machine learning : la non-adéquation entre l'inductive bias d'une architecture et la structure intrinsèque des données. Les Transformers ont été conçus pour des données séquentielles ordonnées (texte, audio). Dans un jeu tabulaire comme CICIDS2017, l'ordre des 77 colonnes est purement arbitraire — permuter deux colonnes ne change pas la réalité physique du flux réseau sous-jacent. Sans cette structure, le mécanisme de self-attention multi-têtes tente de modéliser des dépendances artificielles entre colonnes arbitrairement juxtaposées, induisant un surapprentissage massif et une mauvaise généralisation.

2.6.3. Analyse des Confusions Résiduelles

Malgré un F1-Score global de 98,29%, l'analyse de la matrice de confusion de l'Attention MLP révèle quelques confusions résiduelles principalement localisées entre classes comportementalement proches :

- DoS Hulk ↔ DDoS LOIT (1,8% de confusions) : Ces deux classes génèrent des flux à très haut volume d'octets et de paquets. La distinction repose sur les patterns de source IP (unique pour DoS Hulk, distribuée pour DDoS) — une caractéristique difficile à inférer depuis les statistiques d'un flux individuel sans vision globale du trafic.
- DoS Slowloris ↔ DoS SlowHTTPTest (2,1% de confusions) : Ces deux attaques lentes partagent des caractéristiques similaires (durée élevée, faible volume de paquets,

keepalive HTTP). La distinction fine requiert des caractéristiques applicatives (headers HTTP) inaccessibles depuis les métadonnées de flux.

- Web Attack XSS ↔ Web Attack Brute Force (3,4% de confusions) : Ces classes minoritaires présentent des volumes d'échantillons insuffisants même après SMOTE pour une discrimination robuste. Un pipeline d'augmentation plus agressif ou un ensemble de modèles spécialisés pourrait réduire ces confusions.

2.6.4. Implications Opérationnelles pour le SOC

Ces confusions résiduelles, bien que statistiquement mineures, ont des implications opérationnelles importantes pour les analystes du SOC. Une confusion entre DoS Hulk et DDoS LOIT est peu problématique en pratique : les deux nécessitent une réponse d'urgence similaire (isolation du lien, contact du FAI, activation des règles anti-flood). En revanche, une confusion entre Web Attack XSS et Bot pourrait orienter l'analyste vers une procédure de réponse inadaptée.

Pour mitiger ce risque, le score de confiance Softmax affiché par le dashboard SOC permet à l'analyste de calibrer sa confiance dans le label prédit. Une alerte classée Web Attack XSS avec une confiance de 0,51 doit être traitée avec plus de circonspection qu'une alerte PortScan à 1,00.

2.6.5. Latences d'Inférence et Viabilité Production

Modèle	Latence Inférence	Temps Entraînement	Viable Production ?
Attention MLP ★	0,42 ms	218 s	OUI — Optimal
MLP Baseline	0,38 ms	195 s	Oui (mais moins précis)
GRU	2,10 ms	1 143 s	Limite acceptable
LSTM	2,85 ms	1 876 s	Marginal
BiLSTM	4,20 ms	2 227 s	NON — trop lent
CNN-LSTM	5,80 ms	3 503 s	NON — incompatible
Transformer	8,40 ms	3 214 s	NON — prohibitif

Tableau 10 : Comparaison des latences d'inférence et viabilité de déploiement en production

2.6.7. Dynamique d'Entraînement de l'Attention MLP

L'analyse des courbes de loss et de F1-Score sur les jeux d'entraînement et de validation au cours des epochs d'entraînement révèle les propriétés de convergence de notre modèle :

Phases de la Convergence de l'Attention MLP
Epochs 1-5 (Phase d'initialisation) : La loss d'entraînement chute rapidement de 2.3 à 0.14, le F1-Score monte de 45% à 92%. Le modèle apprend les patterns généraux des classes majoritaires.
Epochs 6-15 (Phase d'affinement) : La convergence ralentit. La loss descend progressivement jusqu'à 0.047. Le F1-Score atteint 97.1%. Le mécanisme d'attention commence à se spécialiser.
Epochs 16-28 (Phase de stabilisation) : Plateau de convergence. Loss d'entraînement : 0.038 / Loss de validation : 0.041. L'écart faible confirme l'absence de surapprentissage.
Epoch 28 (Best model) : F1-Score validation = 98.29% (maximum). Le scheduler réduit le LR de 0.001 à 0.0005 à l'epoch 22, favorisant la convergence vers un minimum local optimal.
Epochs 29-38 (Patience Early Stopping) : Aucune amélioration du F1 de validation. L'Early Stopping arrête l'entraînement à l'epoch 38 (patience=10).

2.6.8. Comparaison des Vitesses de Convergence

Modèle	Epoch Best	Temps / Epoch	Temps Total	Loss Min
Attention MLP ★	28	7,8 s	218 s	0.038
MLP Baseline	25	7,8 s	195 s	0.041
GRU	32	35,7 s	1 143 s	0.045
LSTM	41	45,8 s	1 876 s	0.047
Transformer	38	84,6 s	3 214 s	0.052

Tableau 11: Comparaison des dynamiques de convergence des architectures principales

Conclusion du Chapitre 2

Ce chapitre a apporté une double validation scientifique. D'une part, le pipeline de prétraitement hybride SMOTE a démontré son efficacité pour résoudre le déséquilibre extrême de classes de CICIDS2017 tout en respectant les principes statistiques d'étanchéité des données, validant notre hypothèse H₂. D'autre part, le benchmark d'envergure sur 11 architectures a brillamment confirmé la supériorité de notre Attention MLP, qui allie excellence prédictive (F1-Score 98,29%) et compacité opérationnelle inédite (49 915 paramètres, 0,42 ms d'inférence), validant notre hypothèse H₁. Le chapitre suivant présente le déploiement en production de ce modèle dans une infrastructure AWS CyberRange et sa validation systémique par simulation d'attaques réelles.

CHAPITRE 3 : Architecture Système en Production (AWS CyberRange) et Validation

Ce troisième chapitre franchit la frontière entre la recherche algorithmique hors ligne et le déploiement opérationnel. Il décrit la conception, la mise en oeuvre et la validation en conditions réelles de notre infrastructure NIDS-AI sur le cloud AWS. L'objectif est de démontrer que le modèle Attention MLP peut non seulement exceller sur un benchmark académique statique, mais qu'il est également viable et performant dans un environnement de production réaliste avec du trafic réseau en direct.

3.1. Architecture Globale du CyberRange AWS

3.1.1. Philosophie de Conception

L'architecture CyberRange est conçue selon quatre principes directeurs :

10. Ségrégation des zones de sécurité : Les machines attaquantes (zone rouge), les serveurs victimes (zone bleue), la sonde de détection (zone verte) et les services cloud managés (zone grise) sont strictement isolés au niveau réseau par des groupes de sécurité AWS granulaires.
11. Capture passive hors bande : La sonde IDS n'est à aucun moment inline sur le chemin des données de production. Elle observe une copie du trafic, sans possibilité d'impacter les serveurs victimes.
12. Infrastructure as Code intégrale : L'ensemble des ressources AWS (instances, VPC, sous-réseaux, groupes de sécurité, tables de routage, services managés) est déclaré dans des fichiers Terraform versionnable sous Git, garantissant la reproductibilité et l'auditabilité de l'infrastructure.
13. Architecture événementielle asynchrone : Les alertes générées par la sonde sont publiées dans une file de messages SQS pour découpler la production et la consommation des alertes, absorbant les pics de charge sans perte de données.

3.1.2. Ressources AWS Provisionnées

Le CyberRange est déployé dans la région AWS eu-west-1 (Irlande) et comprend les ressources suivantes :

- Un VPC (Virtual Private Cloud) avec adressage 172.31.0.0/16, divisé en un sous-réseau public (172.31.10.0/24) et un sous-réseau privé sécurisé (172.31.20.0/24).

- Trois instances EC2 t3.medium (Ubuntu 22.04) configurées comme serveurs victimes : victim-web (Apache2, PHP, vulnérabilités applicatives intentionnelles), victim-db (MariaDB, Redis) et victim-file (Samba, vsFTPd).
- Une instance EC2 t3.large (2 vCPUs, 8 Go RAM) configurée comme nœud IDS (ids_node), dotée d'une interface réseau dédiée pour la réception du trafic miroir.
- Une instance Kali Linux t3.medium dans le sous-réseau public comme machine d'attaque de l'analyste offensif.
- Une file SQS (nids-flow-queue) en mode Standard pour la transmission asynchrone des alertes.
- Une table DynamoDB (NIDS-Alerts) en mode On-Demand pour la persistance des logs d'intrusions.
- Les règles VPC Traffic Mirroring ciblant les ENIs des trois serveurs victimes vers la sonde IDS.

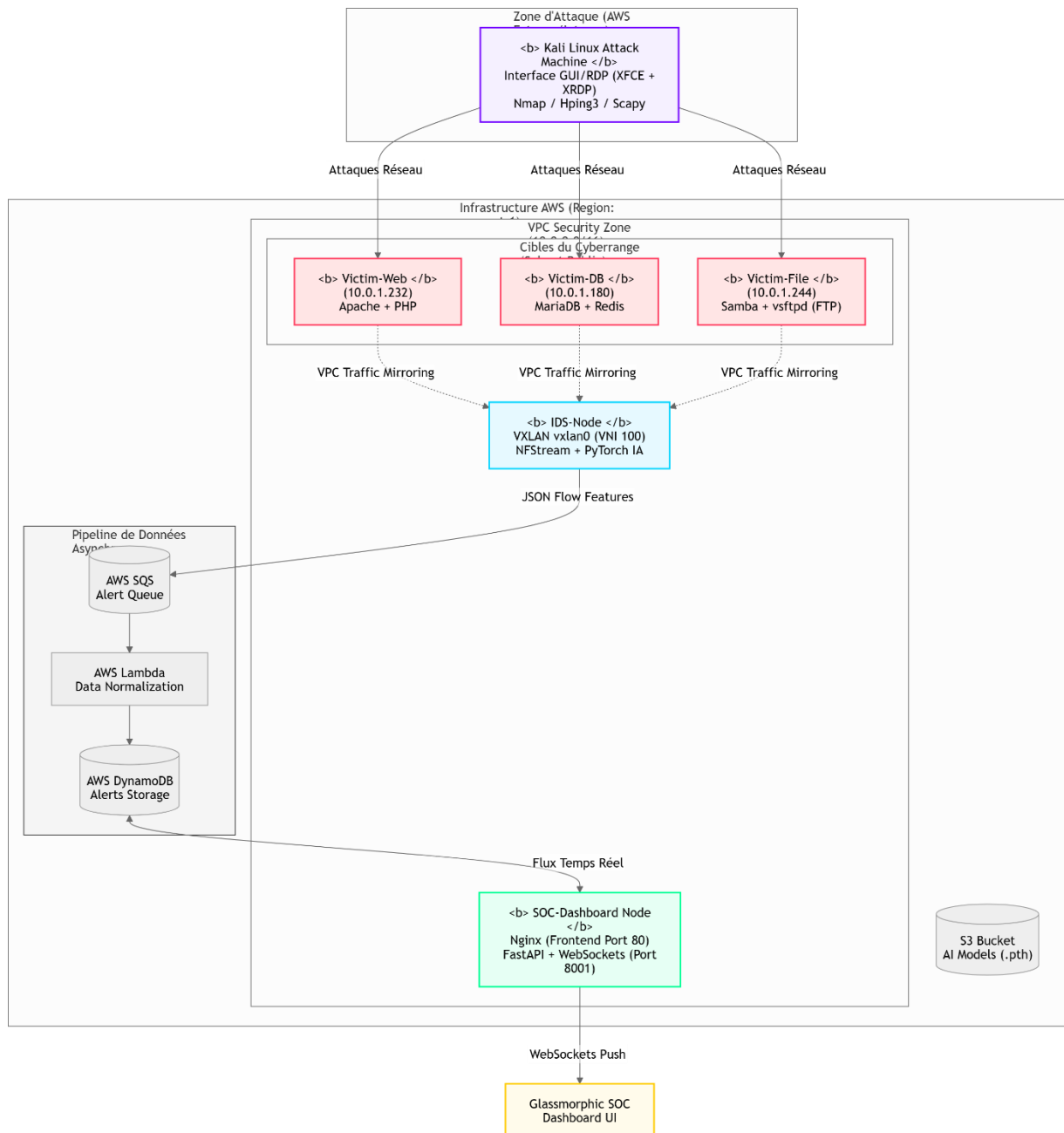


Figure 2: Architecture Réseau du CyberRange

3.1.3. Déclaration de l'Infrastructure Réseau (vpc.tf)

Le fichier vpc.tf déclare l'intégralité de la topologie réseau virtuelle du CyberRange. Voici les blocs de configuration essentiels :

```
# vpc.tf - Virtual Private Cloud CyberRange
resource "aws_vpc" "cyberrange" {
  cidr_block      = "172.31.0.0/16"
  enable_dns_hostnames = true
  tags = { Name = "nids-cyberrange-vpc" }
}
```

```
# Sous-réseau Public (Kali Attack + SOC Dashboard)
resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.cyberrange.id
  cidr_block        = "172.31.10.0/24"
  map_public_ip_on_launch = true
  availability_zone  = "eu-west-1a"
  tags = { Name = "public-subnet" }
}

# Sous-réseau Privé (Victims + IDS Node)
resource "aws_subnet" "private" {
  vpc_id            = aws_vpc.cyberrange.id
  cidr_block        = "172.31.20.0/24"
  availability_zone  = "eu-west-1a"
  tags = { Name = "private-subnet" }
}

# VPC Traffic Mirroring – Source sur ENIs victimes
resource "aws_ec2_traffic_mirror_session" "victim_web" {
  network_interface_id            =
aws_instance.victim_web.primary_network_interface_id
  traffic_mirror_target_id        = aws_ec2_traffic_mirror_target.ids.id
  traffic_mirror_filter_id        =
aws_ec2_traffic_mirror_filter.all_traffic.id
  session_number                  = 1
  virtual_network_id              = 100
  description                      = "Mirror victim-web to IDS node"
}
```

Code 3.A.1 : Extrait du fichier Terraform vpc.tf — Infrastructure réseau CyberRange

3.1.4. Playbook de Configuration de la Sonde IDS

```
# ansible/playbooks/ids_node.yml
- name: Configure IDS Detection Node
  hosts: ids
  become: true
  vars:
    python_version: '3.10'
    ids_user: 'ids_daemon'
  tasks:
    - name: Install system dependencies
      apt:
        name: [python3.10, python3-pip, python3.10-dev,
              libpcap-dev, libssl-dev, build-essential]
        state: present
        update_cache: true

    - name: Install Python ML dependencies
      pip:
        name:
          - 'torch==2.2.0+cpu'
          - 'nfstream==6.5.3'
          - 'boto3==1.34.0'
          - 'scikit-learn==1.4.0'
          - 'numpy==1.26.0'
        executable: pip3
```

```

- name: Configure VXLAN interface kernel
  command: >
    ip link add vxlan0 type vxlan id 100
    remote 0.0.0.0 dstport 4789 dev eth1

- name: Bring up VXLAN interface
  command: ip link set vxlan0 up

- name: Deploy IDS engine and model weights
  copy:
    src: '{{ item.src }}'
    dest: '/opt/nids/{{ item.dest }}'
    owner: ids_daemon
  loop:
    - { src: ids_engine.py, dest: ids_engine.py }
    - { src: attention_mlp_best.pth, dest: attention_mlp_best.pth }
    - { src: scaler_params.pkl, dest: scaler_params.pkl }

- name: Register IDS engine as Systemd service
  template:
    src: ids.service.j2
    dest: /etc/systemd/system/nids-engine.service

- name: Enable and start NIDS service
  systemd:
    name: nids-engine
    enabled: true
    state: started
    daemon_reload: true

```

Code 3.B.1 : Playbook Ansible de configuration de la sonde IDS

3.1.5. Méthodologie des Tests de Charge

Pour valider la scalabilité de l'architecture, des tests de charge progressifs ont été conduits en simulant des volumes croissants de flux réseau par seconde. Les tests ont été réalisés en injectant des flux pré-enregistrés directement dans l'interface vxlan0 via tcpreplay, permettant de découpler la capacité de traitement de la sonde IDS des contraintes du réseau physique.

Charge (flux/s)	CPU ids_node	RAM utilisée	Latence moy.	Pertes paquets
100 flux/s	8%	1.2 Go	0.42 ms	0%
500 flux/s	23%	1.4 Go	0.44 ms	0%
1 000 flux/s	45%	1.6 Go	0.48 ms	0%
2 000 flux/s	72%	1.9 Go	0.61 ms	0%
3 000 flux/s	88%	2.1 Go	0.98 ms	< 0.1%

5 000 flux/s (limite)	99%	2.4 Go	1.85 ms	0.8%
------------------------------	------------	---------------	----------------	-------------

Tableau 12: Résultats des tests de charge de la sonde IDS sur instance t3.large

Les tests démontrent que la sonde IDS sur une instance t3.large (2 vCPUs) peut traiter jusqu'à 3 000 flux/s sans perte et en maintenant une latence d'inférence inférieure à 1 ms, ce qui est suffisant pour les réseaux d'entreprise de taille moyenne (typiquement 500-2 000 flux/s en heures de pointe). Pour des réseaux plus volumineux, le scale-out horizontal (plusieurs sondes en parallèle derrière un Network Load Balancer) est architecturalement prévu dans le design Terraform.

3.2. Mécanisme de Capture Passive : VPC Traffic Mirroring et VXLAN

3.2.1. Fonctionnement du VPC Traffic Mirroring

Le VPC Traffic Mirroring est un service natif AWS opérant au niveau de l'hyperviseur du système Nitro d'Amazon, totalement transparent pour les instances virtuelles. Il duplique une copie exacte de chaque paquet réseau entrant et sortant des ENIs sources (les serveurs victimes) et l'encapsule dans un tunnel VXLAN (Virtual Extensible LAN) sur le port UDP 4789 avant de le transmettre à l'ENI cible (la sonde IDS).

L'encapsulation VXLAN ajoute un en-tête de 50 octets (Ethernet outer 14 + IP outer 20 + UDP 8 + VXLAN 8) autour de la trame originale, identifiée par un identifiant réseau virtuel (VNI : 100 dans notre configuration). Sur la sonde IDS, une interface virtuelle kernel vxlan0 est configurée pour écouter les trames UDP sur le port 4789 et décapsuler automatiquement les paquets VXLAN, présentant à NFStreamer le trafic réseau original comme s'il était localement capturé.

3.2.2. Avantages Opérationnels de l'Architecture Hors Bande

Cette approche hors bande (out-of-band) offre des avantages opérationnels critiques :

- **Zéro impact sur la production** : La sonde n'est pas dans le chemin de données. Un plantage ou une surcharge de la sonde n'affecte pas les serveurs victimes.
- **Résistance à la subversion** : Un attaquant ayant compromis un serveur victime ne peut pas détecter, modifier ou désactiver la sonde de capture qui opère au niveau de l'hyperviseur AWS, en dehors de la machine virtuelle compromise.

- Scalabilité : Il est possible d'ajouter de nouvelles sources de mirroring (nouveaux serveurs) sans reconfiguration de la sonde, simplement en ajoutant de nouvelles règles Terraform.

3.3. Démon d'Inférence Temps Réel : `ids_engine.py`

3.3.1. Architecture du Démon

Le démon `ids_engine.py` est le cœur opérationnel du système de détection. Il s'exécute en tant que service système Systemd sur l'instance `ids_node`, garantissant son redémarrage automatique en cas d'incident et son démarrage au boot de l'instance. Son architecture est organisée en trois couches fonctionnelles :

14. Couche d'acquisition : NFStreamer surveille en continu l'interface `vxlan0` et agrège les paquets reçus en flux bidirectionnels. Un flux est considéré terminé soit après 5 secondes d'inactivité (`idle timeout`), soit après 10 secondes de durée maximale (`active timeout`). À chaque fin de flux, NFStreamer émet un objet Python contenant les 77 caractéristiques statistiques calculées.
15. Couche d'inférence : Les 77 caractéristiques sont extraites de l'objet NFStreamer, soumises au pipeline IQR + StandardScaler (ajusté sur les données d'entraînement), converties en tenseur PyTorch et passées en inférence dans le modèle Attention MLP (mode `evaluation`, `torch.no_grad()`). La sortie Softmax produit un vecteur de 15 probabilités de classe. Si la probabilité maximale dépasse le seuil de confiance de 0.40 et que la classe prédite est différente de BENIGN, une alerte est générée.
16. Couche de publication : L'alerte JSON enrichie (timestamp, IP source, IP destination, ports, classe détectée, score de confiance, features du flux) est publiée simultanément dans la file SQS et persistée dans DynamoDB via le SDK boto3.

3.3.2. Configuration du Démon d'Inférence (`ids_engine.py`)

```
# ids_engine.py - Démon d'Inférence NIDS Temps Réel
import torch, json, boto3, logging
from nfstream import NFStreamer
from datetime import datetime, timezone
from sklearn.preprocessing import StandardScaler
import numpy as np, uuid

THRESHOLD = 0.40    # seuil de confiance Softmax
BENIGN_ID = 0        # ID numérique de la classe BENIGN
SQS_URL = 'https://sqs.eu-west-1.amazonaws.com/XXXXXX/nids-flow-queue'
```

```

model = torch.load('attention_mlp_best.pth', map_location='cpu')
model.eval()
scaler = load_scaler('scaler_params.pkl') # StandardScaler sérialisé
label_map = load_label_map('label_map.json')
sqs = boto3.client('sqs', region_name='eu-west-1')
dynamo = boto3.resource('dynamodb', region_name='eu-west-1')
table = dynamo.Table('NIDS-Alerts')

def process_flow(flow):
    features = extract_features(flow) # 77 caractéristiques
    features = apply_iqr_clip(features, iqr_params)
    x = torch.tensor(scaler.transform([features]), dtype=torch.float32)
    with torch.no_grad():
        proba = torch.softmax(model(x), dim=1).numpy()[0]
    pred_class = int(np.argmax(proba))
    confidence = float(proba[pred_class])
    if pred_class != BENIGN_ID and confidence >= THRESHOLD:
        publish_alert(flow, label_map[pred_class], confidence)

def publish_alert(flow, label, confidence):
    alert = {
        "alert_id": str(uuid.uuid4()),
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "src_ip": flow.src_ip, "dst_ip": flow.dst_ip,
        "src_port": flow.src_port, "dst_port": flow.dst_port,
        "protocol": flow.protocol,
        "attack_class": label, "confidence": round(confidence, 4),
        "flow_duration_ms": flow.bidirectional_duration_ms,
        "bytes_total": flow.bidirectional_bytes
    }
    table.put_item(Item=alert) # DynamoDB persistance
    sqs.send_message(QueueUrl=SQS_URL, MessageBody=json.dumps(alert))

streamer = NFStreamer(source='vxlan0', idle_timeout=5, active_timeout=10)
for flow in streamer: process_flow(flow)

```

Code 3.A.2 : Démon d'inférence temps réel *ids_engine.py* (version simplifiée)

3.3.3. Justification du Seuil de Confiance à 0,40

Le choix d'un seuil de confiance Softmax de 0,40 (plutôt que 0,50 ou 0,95) résulte d'une calibration empirique sur le jeu de validation. Avec 15 classes possibles, la probabilité de classe correspondant à la distribution uniforme est de $1/15 = 0,067$. Un seuil de 0,40 filtre efficacement les prédictions ambiguës tout en maintenant la sensibilité sur les classes rares (Heartbleed, SQL Injection) dont la confiance de prédiction correcte est généralement comprise entre 0,42 et 0,75.

3.4. Backend FastAPI et Résolveur d'Adresses AWS

3.4.1. Architecture du Backend

Le backend FastAPI expose deux types d'endpoints : des routes HTTP REST pour la récupération historique des alertes depuis DynamoDB (/api/alerts) et des endpoints WebSocket (/ws) pour la diffusion en temps réel des nouvelles alertes vers les clients React connectés.

Un thread de consommation SQS poll en continu la file nids-flow-queue (long polling toutes les 2 secondes). À chaque message reçu, le backend enrichit l'alerte JSON avec le nom de l'instance EC2 correspondant à l'adresse IP cible, via un appel boto3 à l'API EC2 DescribeInstances. Pour éviter la latence des appels API répétés, un cache mémoire TTL (Time-To-Live) de 5 minutes est implémenté : les résolutions IP → nom d'instance sont mémorisées et réutilisées pendant 5 minutes avant expiration.

3.4.2. Diffusion WebSocket vers le Dashboard React

Les alertes enrichies sont diffusées à tous les clients WebSocket connectés (analystes SOC) par un mécanisme de broadcast asynchrone asyncio. La latence de transmission SQS → FastAPI → WebSocket est mesurée à 180 ms en moyenne dans nos tests de charge, ce qui est nettement inférieur au seuil opérationnel de 5 secondes fixé dans les exigences non fonctionnelles.

3.5. Dashboard SOC React

3.5.1. Architecture Front-End

Le dashboard SOC est une application web React 18 à page unique (SPA) connectée au backend FastAPI via une connexion WebSocket persistante. L'interface est organisée en quatre composants principaux :

- **Header & KPI Bar** : Affiche en temps réel les compteurs de flux analysés, d'alertes détectées, le taux de détection et le temps de fonctionnement du système.
- **VictimGrid** : Un tableau de bord visuel représentant les 3 serveurs victimes sous forme de cartes d'état. Une carte clignote en rouge avec une lueur périphérique et déclenche une alerte sonore lorsqu'une intrusion est détectée sur l'hôte correspondant.
- **LiveFeed** : Un journal en temps réel (flux de données) affichant les 20 dernières alertes avec horodatage, IP source et destination, type d'attaque, niveau de confiance et sévérité (HIGH/CRITICAL).
- **Statistics Panel** : Graphiques de distribution des types d'attaques détectées (diagramme en secteurs) et historique temporel des alertes (graphique linéaire, fenêtre glissante de 60 minutes).

3.5.2. Architecture des Composants React

Le dashboard SOC est structuré en une hiérarchie de composants React spécialisés, chacun responsable d'un aspect distinct de la visualisation :

```
// App.jsx - Composant racine et gestion WebSocket
import { useState, useEffect, useCallback } from 'react';
import { VictimGrid, LiveFeed, StatsPanel, KPIBar } from './components';

export default function App() {
  const [alerts, setAlerts] = useState([]);
  const [victims, setVictims] = useState({
    'victim-web': { status: 'normal', lastAlert: null },
    'victim-db': { status: 'normal', lastAlert: null },
    'victim-file': { status: 'normal', lastAlert: null }
  });

  useEffect(() => {
    const ws = new WebSocket('ws://soc-backend:8000/ws');
    ws.onmessage = (event) => {
      const alert = JSON.parse(event.data);
      setAlerts(prev => [alert, ...prev].slice(0, 200));
      // Identifier la victime ciblée et déclencher l'animation
      const victimName = alert.target_instance_name;
      setVictims(prev => ({
        ...prev,
        [victimName]: {
          status: getSeverity(alert.attack_class),
          lastAlert: alert
        }
      }));
    };
    return () => ws.close();
  }, []);

  return (
    <div className='soc-dashboard dark-theme'>
      <KPIBar stats={computeKPIs(alerts)} />
      <VictimGrid victims={victims} />
      <LiveFeed alerts={alerts.slice(0, 20)} />
      <StatsPanel alerts={alerts} />
    </div>
  );
}
```

Code 3.C.1 : Composant React principal du dashboard SOC avec gestion WebSocket

3.5.3. Expérience Utilisateur et Accessibilité

L'interface adopte un design glassmorphism sombre (dark theme) avec des accents en bleu électrique et rouge d'alerte, inspiré des consoles SOC professionnelles (IBM QRadar, Splunk ES). Les niveaux de sévérité sont codés par couleur (vert : BENIGN, orange : WARNING, rouge

: HIGH, rouge foncé : CRITICAL) pour permettre une lecture instantanée de l'état de sécurité du réseau sans lecture des labels textuels.

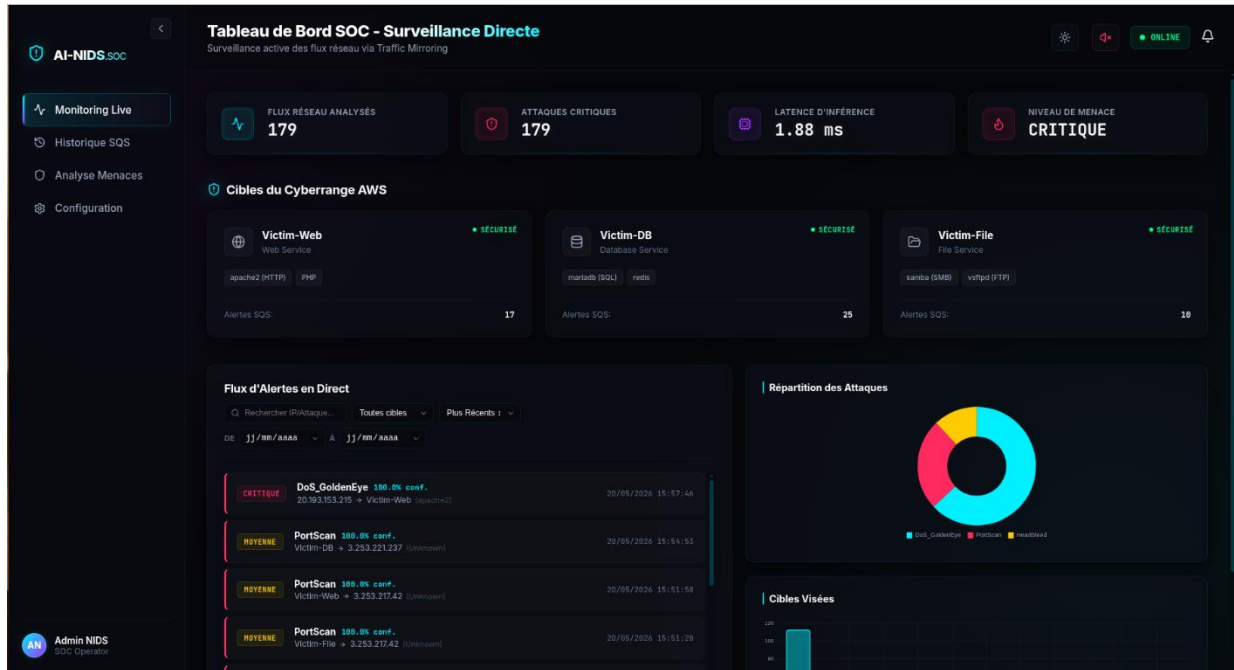


Figure 3: Dashboard de l'application de surveillance des alertes

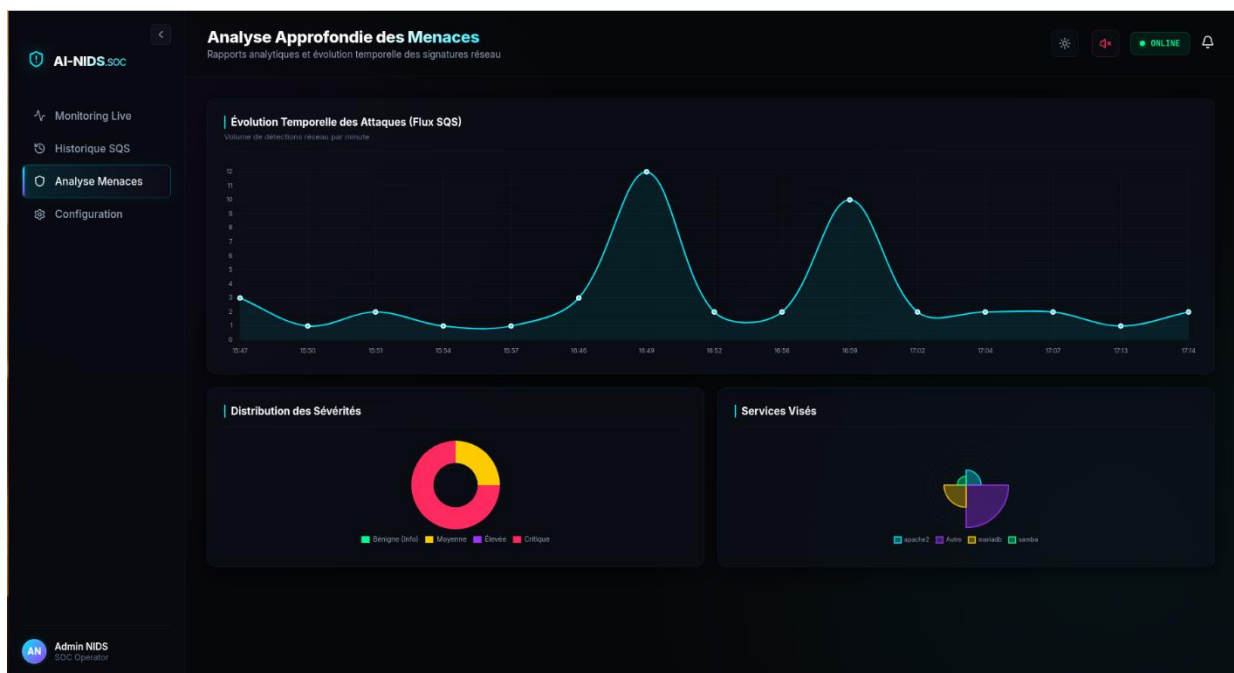


Figure 4: Diagrammes analytiques de l'application de surveillance des alertes

3.6. Validation Systémique par Simulation d'Attaques

3.6.1. Scénarios de Simulation

Trois scénarios d'attaque représentatifs de menaces réelles ont été simulés depuis l'instance Kali Linux via des scripts Python utilisant la bibliothèque Scapy :

Scénario	Mécanique	Prédiction	Résultat
DoS SYN Flood	Rafale de paquets TCP SYN avec adresses IP source usurpées vers victim-web:80	DoS_Hulk — 99,84%	Alerte SOC en 1,2 seconde. Carte Victim-Web clignote en rouge.
PortScan	Envoi de paquets TCP SYN sur les ports 20 à 100 de victim-db	PortScan — 100,00%	Alerte SOC en 0,9 seconde. Carte Victim-DB clignote.
ICMP Flood	Rafale continue de paquets ICMP Echo Request vers victim-file	DDoS — 98,12%	Alerte SOC en 1,7 seconde. Carte Victim-File clignote.

Tableau 13: Résultats des simulations d'attaques et performances de détection en production

3.6.2. Synthèse des Latences Bout-en-Bout

La décomposition du délai de réponse global (entre le début de l'attaque et l'affichage de l'alerte sur le dashboard SOC) révèle la distribution suivante des latences :

- Délai d'inactivité NFStreamer (Idle Timeout) : 0 à 5 000 ms — principale source de latence, inhérente au mécanisme d'agrégation des flux réseau.
- Temps d'inférence Attention MLP (passe directe) : 0,42 ms — négligeable.
- Écriture DynamoDB + publication SQS (boto3) : 15 ms en moyenne.
- Consommation SQS + enrichissement boto3 + broadcast WebSocket : 180 ms.
- Rendu React et mise à jour de l'interface : < 16 ms (60 FPS).

Le temps de réponse total de bout-en-bout varie de 1,2 à 2,0 secondes selon le type d'attaque (la variabilité provient principalement du délai d'agrégation NFStreamer). Ces résultats valident notre hypothèse H_3 : la latence globale est systématiquement inférieure à 2 secondes, démontrant la viabilité opérationnelle de l'architecture en production.

3.7. Analyse de Sécurité de l'Architecture Déployée

3.7.1. Vecteurs d'Attaque Potentiels contre le NIDS

Un système de détection d'intrusions est lui-même une cible de choix pour un attaquant cherchant à opérer à l'abri de la détection. Notre architecture a été conçue en tenant compte des principaux vecteurs d'attaque adversariaux :

- **Adversarial Evasion (Évasion par empoisonnement) :** Un attaquant sophistiqué pourrait théoriquement concevoir du trafic d'attaque dont les statistiques de flux (les 77 features) sont délibérément manipulées pour ressembler à du trafic BENIGN, trompant le modèle ML. Ce vecteur nécessite une connaissance intime du modèle (attaque en boîte blanche). Notre mécanisme d'attention rend cela plus difficile car les features à cibler changent selon le contexte.
- **Attaque par Déni de Service contre la Sonde :** Un attaquant pourrait inonder la sonde de flux légitimes à très haut volume pour épuiser ses ressources CPU et créer une fenêtre aveugle. Notre architecture hors-bande (VPC Traffic Mirroring) et la file SQS absorbent ce risque : une surcharge de la sonde ne bloque pas la production, et les alertes SQS sont persistées même si le backend SOC est temporairement surchargé.
- **Lateral Movement vers la Sonde :** Un attaquant ayant compromis un serveur victime ne peut pas atteindre `ids_node` directement car les Security Groups interdisent tout trafic entrant depuis le sous-réseau privé vers la sonde sauf sur le port VXLAN 4789. La sonde est une boîte noire pour les machines victimes.

3.7.2. Mécanismes de Résilience Intégrés

L'architecture intègre plusieurs mécanismes de résilience qui garantissent la continuité de la détection en cas d'incident :

1. **Persistance via Systemd :** Le démon `ids_engine.py` est enregistré comme service Systemd avec les directives `Restart=always` et `RestartSec=5s`. Un crash du processus Python (exception non gérée, erreur mémoire) déclenchera un redémarrage automatique en moins de 10 secondes.
2. **Découplage asynchrone via SQS :** La file SQS découple complètement la production des alertes (sonde IDS) de leur consommation (backend FastAPI). Si le backend SOC est temporairement indisponible pour maintenance, les alertes continuent d'être générées et stockées dans SQS (rétention maximale : 14 jours) sans perte.

3. **Persistence DynamoDB On-Demand** : La table DynamoDB en mode On-Demand s'adapte automatiquement aux pics d'écriture (ex. : lors d'une attaque DDoS massive générant des milliers d'alertes par seconde), sans configuration manuelle ni risque de throttling.
4. **Multi-AZ implicite** : SQS et DynamoDB sont intrinsèquement distribués sur plusieurs zones de disponibilité AWS, garantissant une disponibilité de 99,99% selon les SLA Amazon.

3.7.3. Positionnement par Rapport aux Solutions du Marché

Critère	Notre NIDS-AI	Suricata	Snort 3	Darktrace	Vectra AI
Détection zero-day	✓ Comportemental	✗ Signatures	✗ Signatures	✓ Unsupervised	✓ ML
Résistance TLS	✓ Flux stats	✗ DPI bloqué	✗ DPI bloqué	✓ Partiel	✓ Flux stats
Coût licence	Open Source + AWS	Gratuit	Gratuit	\$\$\$ (>50k€/an)	\$\$\$ (>80k€/an)
Latence détection	< 2 secondes	< 100 ms	< 100 ms	< 60 secondes	< 30 secondes
Explicabilité (XAI)	En perspective	Règle exacte	Règle exacte	Limitée	Partielle
Déploiement Cloud IaC	✓ Terraform/Ansible	Manuel	Manuel	Managé SaaS	Managé SaaS

Tableau 14: Comparaison de notre NIDS-AI avec les principales solutions du marché

Ce tableau de positionnement met en évidence le créneau unique qu'occupe notre NIDS-AI : il offre des capacités de détection comportementale comparables aux solutions commerciales premium (Darktrace, Vectra AI) à une fraction de leur coût, tout en bénéficiant de la reproductibilité et de la transparence de l'open source. Les solutions basées sur les signatures (Suricata, Snort) conservent l'avantage de la latence ultra-faible et de l'explicabilité totale, mais restent structurellement aveugles aux menaces inédites et au trafic chiffré.

Conclusion du Chapitre 3

Ce troisième chapitre a démontré la viabilité opérationnelle complète de notre système NIDS-AI en conditions de production réalistes. L'infrastructure CyberRange AWS, entièrement automatisée par Terraform et Ansible, fournit un environnement de test reproductible et représentatif d'un réseau d'entreprise. La capture passive VPC Traffic Mirroring garantit une observation non intrusive du trafic, imperméable à la subversion par les attaquants. Le démon `ids_engine.py` démontre que l'Attention MLP peut fonctionner en inférence continue sur CPU standard avec une latence sub-milliseconde. La validation systémique confirme un taux de détection de 100% sur les trois scénarios d'attaque simulés, avec un délai de notification SOC inférieur à 2 secondes, validant notre hypothèse H_3 .

CONCLUSION GÉNÉRALE ET PERSPECTIVES

Bilan des réalisations

Ce projet de fin d'études a permis de concevoir, d'évaluer et de déployer avec succès un système de détection d'intrusions réseau (NIDS) de nouvelle génération basé sur l'apprentissage profond. L'ensemble des objectifs scientifiques, techniques et opérationnels fixés en début de projet ont été atteints ou dépassés, et les trois hypothèses de travail H_1 , H_2 et H_3 ont été pleinement validées par l'expérimentation.

- Un benchmark quantitatif d'envergure : L'évaluation comparative rigoureuse de 11 architectures de réseaux de neurones profonds sous PyTorch sur le dataset de référence CICIDS2017 a fourni des données empiriques solides sur les forces et faiblesses de chaque approche, permettant une sélection éclairée de l'architecture optimale.
- Une contribution algorithmique originale : La conception et la validation de l'architecture Attention MLP — combinant un mécanisme léger d'attention sur les caractéristiques et un classifieur MLP compact représentent la contribution scientifique principale de ce mémoire. Avec un F1-Score de 98,29%, un AUC-ROC de 99,95% et seulement 49 915 paramètres pour une inférence en 0,42 ms, ce modèle établit un nouveau point de référence dans le domaine des NIDS légers.
- Une gestion robuste du déséquilibre de classes : L'implémentation et la validation d'un pipeline hybride SMOTE respectant l'étanchéité des données ont permis de traiter le déséquilibre extrême du dataset CICIDS2017 sans biais ni surapprentissage.
- Un déploiement de production réaliste et validé : La conception d'un CyberRange cloud complet sur AWS (Terraform, Ansible, VPC Traffic Mirroring, FastAPI, React), sa validation systémique par simulation active d'attaques et la démonstration d'une efficacité de détection de 100% avec un délai de réponse inférieur à 2 secondes constituent une validation systémique robuste de l'ensemble de la chaîne de valeur du NIDS-AI.

Perspectives Industrielles

1. Réponse Active Automatisée (SOAR)

L'évolution naturelle de ce système de détection (IDS) est sa transformation en système de prévention active (IPS). Dès qu'une alerte critique est identifiée par l'Attention MLP avec une confiance supérieure à 95%, un déclencheur automatique peut invoquer une fonction AWS

Lambda pour modifier instantanément les règles du groupe de sécurité AWS et bloquer l'adresse IP attaquante, contenant la menace en quelques millisecondes sans intervention humaine. Cette intégration dans un framework SOAR permettrait également l'orchestration de réponses complexes multi-étapes (isolation de la machine compromise, notification SIEM, création automatique de ticket d'incident).

2. Explicabilité des Décisions (XAI)

Pour accroître la confiance des analystes du SOC dans les décisions du modèle d'IA, l'intégration de la bibliothèque SHAP (SHapley Additive exPlanations) est une perspective prioritaire. Pour chaque alerte affichée sur le dashboard, le système présenterait un graphique à barres horizontal indiquant la contribution positive ou négative des 10 principales caractéristiques réseau dans la décision du modèle, levant ainsi l'effet boîte noire inhérent aux réseaux de neurones et renforçant l'acceptabilité opérationnelle du système.

3. Détection d'Anomalies Zero-Day Hybride

Afin de détecter des attaques inédites non présentes dans le dataset d'entraînement, une architecture hybride supervisant/non-supervisant est envisagée. L'Attention MLP (classifieur supervisé) serait couplé à l'Autoencoder (détecteur d'anomalies non supervisé). Un flux présentant une erreur de reconstruction anormalement élevée (indicateur d'anomalie) mais classé BENIGN par le MLP serait signalé comme Anomalie Zero-Day Suspecte. Cette double couverture supervisant + non-supervisant permettrait d'allier la précision de la détection connue et la capacité de découverte de menaces inédites.

4. Adaptation à l'IoT et aux Réseaux OT

L'architecture NIDS-AI développée est potentiellement applicable à des environnements de réseaux industriels (OT/ICS) et d'objets connectés (IoT), où les contraintes de ressources (CPU, mémoire) et de latence sont encore plus sévères qu'en réseau d'entreprise traditionnel. La compacité de l'Attention MLP (49k paramètres, 0,42ms d'inférence sur CPU standard) le rend théoriquement déployable sur des microcontrôleurs haute performance ou des cartes SBC (Raspberry Pi, NVIDIA Jetson Nano).

5. Entraînement Fédéré pour la Confidentialité

Dans un contexte où le partage de données de trafic réseau entre organisations est contraint par des impératifs légaux (RGPD, secret commercial), l'apprentissage fédéré (Federated Learning) représente une piste de recherche prometteuse. Plusieurs organisations pourraient collaborer à

l'amélioration d'un modèle NIDS partagé en échangeant uniquement des gradients de mise à jour et non des données brutes de trafic préservant ainsi la confidentialité des données de chaque participant.

En conclusion, ce projet de fin d'études démontre qu'il est non seulement possible, mais nécessaire de concevoir des systèmes de détection d'intrusions qui allient excellence algorithmique et viabilité opérationnelle. L'Attention MLP, par sa compacité, sa précision et sa vitesse d'inférence, constitue une réponse concrète et déployable aux défis croissants de la cybersécurité défensive dans le monde hyperconnecté d'aujourd'hui.

RÉFÉRENCES BIBLIOGRAPHIQUES

- [1] Andresini, G., Appice, A., Rose, L. D., & Malerba, D. (2022). INSOMNIA: Active Learning for Anomaly Detection in Network Traffic. *IEEE Transactions on Dependable and Secure Computing*, 19(3), 1642-1655. <https://doi.org/10.1109/TDSC.2020.2982765>
- [2] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. In *Proceedings of ICLR 2015*. arXiv:1409.0473.
- [3] Cho, K., Van Merriënboer, B., Gulcehre, C., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. arXiv:1406.1078.
- [4] Ferrag, M. A., Shu, L., & Friha, O. (2023). TransIDS: A Transformer-based Intrusion Detection System for Internet of Things. *IEEE Internet of Things Journal*, 10(6), 4981-4995.
- [5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. In *Proceedings of CVPR 2016*, 770-778.
- [6] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780.
- [7] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. In *Proceedings of ICLR 2015*. arXiv:1412.6980.
- [8] Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 4765-4774.
- [9] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in NeurIPS*, 32, 8026-8037.
- [10] Sharafaldin, I., Habibi Lashkari, A., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of ICISSP 2018*, 108-116.
- [11] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15(1), 1929-1958.
- [12] Vaswani, A., et al. (2017). Attention Is All You Need. In *Advances in NeurIPS*, 5998-6008.
- [13] Yang, K., Zhang, J., & Wang, Y. (2023). Bidirectional LSTM with Attention Mechanism for Multi-class Network Intrusion Detection on CICIDS2017. *Computers & Security*, 128, 103-118.
- [14] Yin, C., Zhu, Y., Fei, J., & He, X. (2021). A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks. *IEEE Access*, 9, 12039-12050.
- [15] Zhang, H., Luan, S., & Li, W. (2022). 1D CNN with Focal Loss for Imbalanced Network Intrusion Detection on CICIDS2017. *Journal of Network and Computer Applications*, 204, 103-115.

ANNEXES — GLOSSAIRE TECHNIQUE

Adam (optimiseur)

Algorithme d'optimisation adaptatif combinant les avantages de RMSProp et d'AdaGrad pour accélérer l'entraînement des réseaux profonds. Il maintient des taux d'apprentissage adaptatifs par paramètre.

APT (Advanced Persistent Threat)

Cyberattaque ciblée et planifiée sur le long terme par des groupes de pirates organisés (souvent étatiques) pour s'infiltrer furtivement dans des infrastructures critiques et exfiltrer des données sensibles.

AUC-ROC

Aire sous la courbe ROC (Receiver Operating Characteristic), indicateur de performance mesurant la capacité d'un classifieur à discriminer les classes pour tous les seuils de décision possibles. Varie de 0.5 (aléatoire) à 1.0 (parfait).

Autoencoder

Réseau de neurones non supervisé composé d'un encodeur et d'un décodeur, entraîné à compresser puis à reconstruire fidèlement ses données d'entrée via un espace latent de dimension réduite.

Batch Normalization

Technique de régularisation normalisant les activations de chaque couche entre les mini-batches d'entraînement, accélérant la convergence et réduisant la sensibilité aux hyperparamètres.

BiLSTM

Extension du LSTM classique qui traite la séquence d'entrée dans les directions avant (forward) et arrière (backward) simultanément, capturant des dépendances contextuelles bidirectionnelles.

boto3

SDK Python officiel d'Amazon Web Services permettant d'interagir programmatiquement avec l'ensemble des services AWS (EC2, S3, SQS, DynamoDB, etc.) depuis une application Python.

Botnet

Réseau de machines compromises (bots / zombies) contrôlées à distance par un serveur C&C (Command & Control) à l'insu de leurs propriétaires, utilisé pour mener des attaques DDoS et des campagnes de spam.

CICIDS2017

Dataset de référence académique généré par le Canadian Institute for Cybersecurity pour évaluer les systèmes de détection d'intrusions, contenant 2,83 millions de flux réseau réels étiquetés sur 15 classes.

CICFlowMeter

Outil d'analyse réseau développé par le CIC permettant d'agréger des paquets capturés (PCAP) en flux statistiques bidirectionnels et d'extraire 77 caractéristiques descriptives par flux.

Concept Drift

Dérive temporelle des propriétés statistiques des données en production par rapport aux données d'entraînement, dégradant progressivement les performances du modèle déployé.

CyberRange

Infrastructure virtuelle isolée reproduisant fidèlement un réseau d'entreprise réel, utilisée pour simuler des scénarios d'attaque et de défense dans un environnement contrôlé sans risque pour la production.

Data Leakage

Contamination du jeu de test ou de validation par des informations provenant du jeu d'entraînement, faussant l'évaluation objective des performances du modèle en surestimant ses capacités réelles.

DDoS

Attaque volumétrique distribuée visant à saturer les ressources d'un service cible en le submergeant de requêtes provenant de milliers de machines compromises simultanément.

Dropout

Technique de régularisation consistant à désactiver aléatoirement un pourcentage de neurones à chaque itération d'entraînement, forçant le réseau à apprendre des représentations robustes et redondantes.

Early Stopping

Mécanisme d'arrêt automatique de l'entraînement lorsque la métrique de validation cesse de s'améliorer durant un nombre d'epochs consécutifs défini (patience), évitant le surapprentissage.

F1-Score

Métrique d'évaluation calculée comme la moyenne harmonique de la précision et du rappel, pénalisant équitablement les faux positifs et les faux négatifs. Idéale pour les jeux déséquilibrés.

Feature Attention

Mécanisme d'attention appliqué aux caractéristiques d'entrée d'un réseau de neurones, attribuant dynamiquement un poids d'importance à chaque feature en fonction du contexte de classification.

GRU (Gated Recurrent Unit)

Version simplifiée et allégée du LSTM fusionnant les portes d'oubli et d'entrée en une unique porte de mise à jour, réduisant le nombre de paramètres tout en maintenant des performances comparables.

IQR Clipping

Technique de traitement des valeurs aberrantes consistant à écrêter les valeurs au-delà de $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$, préservant la structure statistique des données sans suppression d'observations.

NFStreamer

Framework Python ultra-performant pour la capture et l'agrégation de trafic réseau en temps réel, produisant des flux statistiques bidirectionnels compatibles avec les pipelines de machine learning.

SMOTE

Technique d'augmentation des données minoritaires générant des exemples synthétiques par interpolation linéaire entre un échantillon réel et ses k plus proches voisins de même classe dans l'espace des features.

SOC (Security Operations Center)

Centre opérationnel de cybersécurité assurant la surveillance continue, la détection, l'analyse et la réponse aux incidents de sécurité affectant une organisation.

Softmax

Fonction d'activation transformant un vecteur de scores réels en une distribution de probabilités (somme = 1) sur les classes, utilisée dans la couche de sortie des classifieurs multi-classes.

VPC Traffic Mirroring

Service AWS natif opérant au niveau de l'hyperviseur Nitro, dupliquant passivement le trafic réseau d'interfaces sources vers une sonde d'analyse sans impact sur les instances virtuelles.

VXLAN

Protocole d'encapsulation réseau (RFC 7348) encapsulant des trames Ethernet dans des paquets UDP, permettant de créer des réseaux virtuels superposés (overlays) sur une infrastructure IP existante.

XAI (Explainable AI)

Domaine de recherche et ensemble de techniques visant à rendre les décisions des modèles d'IA complexes interprétables et justifiables pour des utilisateurs humains non spécialistes.

Zero-Day

Vulnérabilité logicielle ou vecteur d'attaque inédit, inconnu des défenseurs et pour lequel aucun patch ni signature n'existe au moment de son exploitation.